

Anonymous Authentication and Key Agreement, Revisited^{*}

Yanqi Zhao
Xi'an University of
Posts and Telecommunications
zhaoyanqi2019@163.com

Xiangyu Liu[†]
CISPA
xiangyu1994liu@gmail.com

Min Xie
Harbin Institute of
Technology, Shenzhen
minxie@stu.hit.edu.cn

Xiaoyi Yang
Xi'an University of
Posts and Telecommunications
yangxiaoqi@xupt.edu.cn

Jianting Ning
Wuhan University
jtning88@gmail.com

Baodong Qin
Xi'an University of
Posts and Telecommunications
qinbaodong@xupt.edu.cn

Haibin Zhang
Yangtze Delta Region Institute
of Tsinghua University
bchainzhang@aliyun.com

Yong Yu
Shaanxi Normal University
yuyong@snnu.edu.cn

Abstract—In NDSS 2024, Yu et al. proposed AAKA, an Anonymous Authentication and Key Agreement scheme designed to protect users' privacy from mobile tracking by Mobile Network Operators (MNOs). AAKA aims to provide both anti-tracking privacy and traceability (lawful de-anonymization), allowing subscribers to access the network via anonymous proofs while enabling a Law Enforcement Agency (LEA) to trace the real identity if misbehaviors are detected. However, we identify that the AAKA scheme in NDSS 2024 is insecure since the subscriber's identity is exposed within the protocol, thereby failing to achieve the claimed privacy and traceability.

Building on the repair of AAKA, we propose AAKA+, Anonymous Authentication and Key Agreement with Verifier-Local Revocation, a new mobile authentication scheme, to ensure privacy against mobile tracking. In addition to the privacy and traceability introduced in NDSS 2024, AAKA+ additionally allows the MNO to immediately assert whether the associated subscriber has been traced and revoked upon receiving an anonymous proof. We formally define the syntax and the security model of AAKA+ and propose two concrete schemes, AAKA+BB and AAKA+PS, based on the Boneh-Boyen signature and the Pointcheval-Sanders signature schemes, respectively. Both AAKA+BB and AAKA+PS are pairing-free on the user equipment side and compatible with existing cellular infrastructure. Experimental results show that our schemes are practical, with anonymous proof generation taking approximately 18 milliseconds for a constrained device.

Keywords—Anonymous Credential, Anti-Tracking Privacy, Verifier-Local Revocation, Authentication Protocols

I. INTRODUCTION

Mobile tracking: privacy threat and surveillance. Mobile communication systems have become an indispensable part of daily life. However, despite providing unprecedented communication services and connectivity, mobile cellular technology also raises significant privacy concerns due to the exposure of vast amounts of subscriber data [1]. As shown in Fig. 1 (step (3)), Mobile Network Operators (MNOs, HN or SN) can uniquely identify and locate users during their cellular visits and create tracking profiles that record private information, such as location and service requests. This private information

is often at risk of being obtained by insiders or external attackers from the MNO and sold to a third party.

The dilemma between user privacy and liability further complicates the issue of mobile tracking. Location tracking data from mobile carriers, most commonly cellular site location information (CSLI), has been used for a long time by law enforcement agencies (LEAs) to provide evidence in criminal investigations [2]. However, this data collection capability can help unauthorized LEAs or states conduct illegal surveillance. The 3GPP community has been standardizing the lawful interception (LI) processes in cellular architecture [3], [4], aiming to regulate how LEAs access mobile user data.

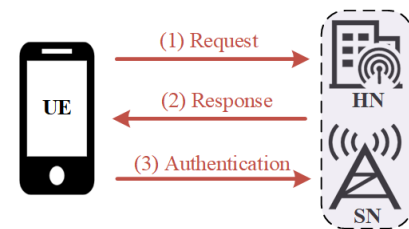


Fig. 1: The Mobile Network: UE, HN, and SN stand for user equipment, home network, and serving network, respectively. A more detailed figure for the architecture is given in Fig. 2.

Verifier-local revocation. Once a user engages in illegal activities, LEA and network providers should be able to trace the user through their previous network requests (known as traceability [5]). It is also crucial to revoke the identities of these illegal users and prevent them from accessing network services [6], [7]. This measure ensures regulatory compliance, protects network integrity, and maintains user trust. By enforcing strict access controls and promptly revoking privileges for unlawful behavior, mobile operators can reduce security risks and provide a reliable environment for legitimate users.

The insecurity and flaws of YZZ+ AAKA. In NDSS 2024, Yu et al. [8] proposed a scheme called AAKA (Anonymous Authentication and Key Agreement, referred to YZZ+ AAKA), aiming to solve the two problems mentioned above: subscriber

^{*}A preliminary version of this paper appears in ACSAC 2025.

[†]Corresponding author.

privacy and traceability. To this end, Yu et al. combined the variant Boneh-Boyen signature scheme in [9] and the idea of anonymous credentials by Camenisch et al. [10], and designed a new anonymous credential scheme with functionality of lawful trace. Though the YZZ+ AAKA scheme in [8] is claimed to guarantee privacy and traceability, unfortunately, there are significant design flaws in YZZ+ AAKA, which means that it cannot achieve privacy, and even correctness as it claimed (see Section III for more details). After several unsuccessful attempts to fix the YZZ+ AAKA scheme, we noticed that it is not merely flawed, but insecure and incomplete. Therefore, the following question remains open:

How to design an (efficient) anonymous authentication and key agreement (AAKA) scheme to guarantee both subscriber privacy and traceability against mobile tracking?

In this work, we revisit this question and provide the first formal treatment of AAKA. Additionally, we address the issue of verifier-local revocation: once an illegal subscriber is traced by LEAs, both the subscriber's identity and relevant tracing information are added to a revocation list. This enables the MNO to identify and reject any future network requests from the subscriber without further interaction with LEAs.

A. Our Contributions

We summarize our contributions as follows.

1) **An attack on YZZ+ AAKA.** We identify that the YZZ+ AAKA (Anonymous Authentication and Key Agreement) scheme [8] is insecure. Specifically, we show in Section III that an MNO can easily extract the identity of the User Equipment (UE), and hence break the privacy.

2) **AAKA+: a formalized and enhanced model.** We propose the concept of Anonymous Authentication and Key Agreement with Verifier-Local Revocation, abbreviated as AAKA+. We formally define the syntax and the security model of AAKA+, which captures the unlinkability, unforgeability, traceability, and pseudorandomness of session keys (a.k.a. anchor keys). In addition, AAKA+ supports verifier-local revocation, enabling MNOs to assert whether a user has been revoked. This addresses the gaps in the formalization of AAKA that were overlooked in [8].

3) **Two practical proposals of AAKA+.** As our main contribution, we propose two concrete schemes, AAKA+BB based on the Boneh-Boyen signature scheme [11], and AAKA+PS based on the Pointcheval-Sanders signature scheme [12]. We formally prove the security in our enhanced model. The experimental results show that our schemes are practical, with anonymous proof generation taking approximately 18 milliseconds for a constrained device.

AAKA+ allows subscribers to access the network anonymously. At the same time, the LEA can trace the subscriber's identity and revoke it if illegal activities have been detected. Any further service request from this revoked subscriber will be rejected. In more detail, AAKA+ accomplishes the following design goals.

• **Anti-tracking privacy (unlinkability).** As a fundamental feature, AAKA+ allows a UE, a paid subscriber, to get network service anonymously. This is achieved through an anonymous

credential proof, in which the identity is hidden and only public attributes (e.g., the expiration date) are revealed.

• **Traceability and verifier-local revocation.** Once an illegal activity is detected, the LEA and the MNO can de-anonymize the anonymous credential proof to trace the real identity of the UE. Additionally, a tracing message is revealed during this process, enabling the MNO to immediately determine whether the UE associated with the proof has been traced and revoked whenever an anonymous credential proof is received.

• **Anti-counterfeiting protection (unforgeability).** The mobile network services are accessible only to paid subscribers. This also implies that the certificates by the HN in AAKA+ are non-transferable.

• **Non-interactive roaming support.** An SN can authenticate unknown UEs without interacting with the HN (the credential issuer).

• **Compatibility and practicality.** The storage and computation overhead on the UE side is minimal. Notably, the UE is not burdened with heavy pairing operations, ensuring that the AAKA+ schemes are compatible with existing cellular infrastructure and standardized SIMs (for example, Java Card).

B. Related Works

The 3GPP consortium has released its fifth-generation (5G) authentication and key agreement protocol, detailed in Technical Specification (TS) 33.501 for mobile communication systems (5G-AKA) [13]. Koutsos proposed the AKA+ protocol [14] against linkability attacks by re-arranging the message flow of 5G-AKA. However, it is impractical to implement AKA+ UE side operation under standardized SIM cards, since SIM cards for all users have to be replaced. Wang et al. proposed 5G-AKA' [15], which utilizes a shared key established based on ECIES-KEM to encrypt the challenges sent to HNs against linkability attacks. Hong et al. [16] revealed that the current GUTI reallocation mechanism can be used to track subscribers' location, and introduced a lightweight and unpredictable GUTI reallocation scheme to guarantee location security. Du et al. [17] explored the privacy threats in 5G and beyond cellular networks, and argued that to protect users' privacy from mobile network operators. Schmitt et al. proposed PGPP [18], which authenticates and accounts for users without identifying them, while utilizing tokens (blind signatures) to defend against malicious MNOs. Yu et al. [8] proposed AAKA to prevent mobile tracking in cellular networks. Unfortunately, [8] provides neither a formal definition nor a formalized security model, and the concrete scheme is insecure. We present the attack against the privacy in Section III.

We refer to Table I for abbreviations used in this paper.

II. PRELIMINARIES

A. Pairing Groups and Assumptions

Let $\Phi = (q, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, g_1, g_2, e)$ be a pairing group such that $\mathbb{G}_1$, $\mathbb{G}_2$, and $\mathbb{G}_T$ are cyclic groups of order q , and g_1, g_2 are generators of $\mathbb{G}_1$ and $\mathbb{G}_2$, respectively. $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ is an efficiently computable bilinear map. In this work, we use the type-3 pairing groups [19], i.e., $\mathbb{G}_1 \neq \mathbb{G}_2$, and there exists no isomorphism between $\mathbb{G}_1$ and $\mathbb{G}_2$.

TABLE I: A table of abbreviations

Abbreviation	Definition
AMF	Access and Mobility Management Function
GUTI	Global Unique Temporary Identifier
HN	Home Network
LEA	Law Enforcement Agency
LI	Lawful Interception
MNO	Mobile Network Operator
OTA	Over-The-Air
SIM	Subscriber Identity Module
SN	Serving Network
SUPI	Subscriber Permanent Identifier
UE	User Equipment
USIM	Universal Subscriber Identity Module

Definition 1 (DDH & SXDH Assumptions [20]). *The Decisional Diffie-Hellman (DDH) assumption holds on $\mathbb{G}_1$, if it is hard to distinguish the distribution $(g_1, g_1^u, g_1^v, g_1^{uv})$ from the distribution $(g_1, g_1^u, g_1^v, g_1^w)$, where $u, v, w \in \mathbb{Z}_q$ are randomly sampled. Given a pairing group Φ , if the DDH assumption holds on both $\mathbb{G}_1$ and $\mathbb{G}_2$, then we say the SXDH assumption holds.*

Definition 2 (k -SCDHI Assumption (over Pairings) [9]). *The k -Strong Computational Diffie-Hellman Inversion (k -SCDHI) assumption holds on a pairing group Φ , if given $(g_2^{x_0}, g_2^{x_1}, \dots, g_2^{x_k})$ with randomly sampled $x_0, \dots, x_k \in \mathbb{Z}_q$, and oracles $(\mathcal{O}_{BB}(\cdot), \{\mathcal{O}_{DH_i}(\cdot)\}_{i=0, \dots, k})$, it is hard to output $((m_1^*, \dots, m_k^*), h \in \mathbb{G}_1)$ such that $h^* = g_1^{\frac{1}{x_0 + \sum_{i=1}^k x_i m_i^*}}$, and $(m_1^*, \dots, m_k^*)$ has not been asked to $\mathcal{O}_{BB}(\cdot)$. Here the oracle $\mathcal{O}_{BB}(\cdot)$ takes as input $(m_1, \dots, m_k) \in \mathbb{Z}_q^k$ and outputs $g_1^{\frac{1}{x_0 + \sum_{i=1}^k x_i m_i}}$, and the oracle $\mathcal{O}_{DH_i}(\cdot)$ ($i \in \{0, 1, \dots, k\}$) takes as input $f \in \mathbb{G}_2$ and outputs f^{x_i} .*

The original k -SCDHI assumption was proposed by Camenisch et al. [9] over a cyclic group $\mathbb{G}$, and proved to be held in the generic group model (GGM) [21]. Here we adapt it to the (asymmetric) pairing setting. We formally prove it in GGM in Appendix A.

B. Zero-knowledge Proof of Knowledge

A zero-knowledge proof of knowledge (ZKPoK) [22], [23] scheme for binary relation R is a protocol between a prover $\mathcal{P}$ and a verifier $\mathcal{V}$ that satisfies:

Completeness. If instance c and witness w satisfies $(c, w) \in R$, and $\mathcal{P}$ and $\mathcal{V}$ execute the protocol honestly, then $\mathcal{V}$ accepts.

Knowledge extractability (a.k.a. validity). There exists an extractor such that for every instance c and every interactive prover $\mathcal{P}^*$ who makes $\mathcal{V}$ accepts with a non-negligible probability, it can extract a witness w satisfying $(c, w) \in R$ with a non-negligible probability.

Zero-knowledge. There exists a zero-knowledge simulator that for every $(c, w) \in R$, given only the instance c , can simulate a transcript that is indistinguishable from an honestly generated one.

Many three-move identification protocols (e.g., the Schnorr protocol [24]) can be transferred into a non-interactive ZKPoK scheme via the Fiat-Shamir paradigm [25] with completeness, (knowledge) extractability, and zero-knowledge. In this

work, we follow the notation introduced by Camenisch and Stadler [26] for proving the knowledge of discrete logarithms over $\mathbb{G}_1$ and $\mathbb{G}_2$. An example is expressed as

$$\Pi : \pi \in \text{ZKPoK}\{(x, y) : X = g_1^x h^y \wedge Y = g_2^y\},$$

where (x, y) is the witness, (g_1, g_2, h, X, Y) is the instance, and π is a non-interactive zero-knowledge proof showing that (x, y) satisfies the statements after the colon.

C. Boneh-Boyen Signature and Its (Multiple-message) Variant

In [9], Camenisch et al. proposed a message authentication code (MAC) scheme based on the (weak) Boneh-Boyen (BB) signature [11] to support multiple messages and the security is based on the k -SCDHI assumption over cyclic group $\mathbb{G}$. It is easy to transfer Camenisch et al.'s MAC scheme to an EUF-CMA secure signature scheme by introducing pairings. We present the (variant) BB (multi-message) signature scheme as follows (we assume basic familiarity of signatures and the unforgeability security).

- **KeyGen**(1^λ): Let $\Phi = (q, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, g_1, g_2, e)$ be a pairing group. Randomly sample $x_0, x_1, \dots, x_k \in \mathbb{Z}_q$ and compute $X_0 = g_2^{x_0}, X_1 = g_2^{x_1}, \dots, X_k = g_2^{x_k}$. Return $(\text{pk}, \text{sk}) = ((X_0, \dots, X_k), (x_0, \dots, x_k))$.
- **Sign**($\text{sk}, (m_1, \dots, m_k)$): Return $\sigma = g_1^{1/(x_0 + \sum_{i=1}^k x_i m_i)}$.
- **Ver**($\text{pk}, (m_1, \dots, m_k), \sigma$): Return 1 if $e(\sigma, X_0 \prod_{i=1}^k X_i^{m_i}) = e(g_1, g_2)$ and 0 otherwise.

III. INSECURITY OF YZZ+ AAKA

In this section, we review the core protocol of YZZ+ AAKA by Yu et al. [8].

The YZZ+ AAKA presentation and verification protocol completes the authentication and key agreement function, of which the 5b sub-protocol of YZZ+ AAKA [8] is shown below.

$$\begin{aligned}
&5b. r, a, b \xleftarrow{R} \mathbb{Z}_p^*, \sigma' \leftarrow \sigma^r \\
&\hat{\sigma}_i \leftarrow \sigma_i^r \text{ for } i = (0, \dots, 4) \\
&\bar{\sigma} \leftarrow \prod_{i=1}^4 \hat{\sigma}_i^{-m_i} g_1^r \\
&(c_1, c_2) \leftarrow (g_1^r, m_4 h^r) \\
&A = \hat{\sigma}_0 \prod_{i=1}^3 \hat{\sigma}_i^{m_i}, B = -c_2 h^{-r} \\
&\pi \in \text{ZKPoK}\{(m_4, r) : A = g_1^r \hat{\sigma}_4^{-m_4} \wedge A = g_1^r \hat{\sigma}_4^B \\
&\quad \wedge -B^{-1} c_2 = h^r \wedge c_1 = g^r\} \\
&\text{Pres} \leftarrow (\{m_i\}_{i=1}^3, \bar{\sigma}, \sigma', \{\hat{\sigma}_i\}_{i=0}^4, c_1, c_2, A, B, \pi)
\end{aligned}$$

In 5b, the UE generates a zero-knowledge proof of (m_4, r) for holding a credential cred, and computes a ciphertext of m_4 under ElGamal encryption. Here $\text{cred} = (\sigma, \sigma_0, \dots, \sigma_4)$, where $\sigma_i = \sigma^{x_i}$ (x_i the secret key(s) of HN), and $\sigma = g_1^{1/(x_0 + \sum_{i=1}^4 x_i m_i)}$ is a variant BB signature satisfying the verification equation $e(\sigma, X_0 \prod_{i=1}^4 X_i^{m_i}) = e(g_1, g_2)$ (see Section II-C). Let h be the public key of LEA. The UE generates $(c_1, c_2) = (g_1^r, m_4 h^r)$, and constructs a zero-knowledge proof π ¹ such that

$$\pi \in \text{ZKPoK}\{(m_4, r) : A = g_1^r \hat{\sigma}_4^{-m_4} \wedge A = g_1^r \hat{\sigma}_4^B \wedge -B^{-1} c_2 = h^r \wedge c_1 = g_1^r\}. \quad (1)$$

¹The equation $A = g_1^r \hat{\sigma}_4^B$ requires that $B \in \mathbb{Z}_p$. However, $-B^{-1} c_2 = h^r$ implies that $B \in \mathbb{G}_1$. Clearly B cannot be an element of both $\mathbb{Z}_p$ and $\mathbb{G}_1$. We guess this is a clerical error in [8].

UE sends $\text{Pres} = (\{m_i\}_{i=1}^3, \bar{\sigma}, \sigma', \{\hat{\sigma}_i\}_{i=0}^4, c_1, c_2, A, B, \pi)^2$ as an anonymous proof to SN. Yu et al. [8] presented the construction of π in Appendix E, and proved the security via Lemma 2. The authors divide the proof π into the following four sub-proofs.

$$\begin{aligned}\pi_1 &\in \text{ZKPoK}\{(m_4, r) : A = g_1^r \hat{\sigma}_4^{-m_4}\} \\ \pi_2 &\in \text{ZKPoK}\{(r) : A = g_1^r \hat{\sigma}_4^B\} \\ \pi_3 &\in \text{ZKPoK}\{(r) : -B^{-1}c_2 = h^r\} \\ \pi_4 &\in \text{ZKPoK}\{(r) : C_1 = g_1^r\}\end{aligned}$$

Obviously, we cannot separate the proof of conjunction paradigms into four “sub-proofs” in this way, since the four expressions share the common witness (m_4, r) (though the concrete scheme of π seems correct). Skipping this fault, we present the construction of π_3 in [8] as follows.

Let $y_3 = -B^{-1}c_2$. UE executes the following steps:

- 1) Randomly takes $a_3 \in Z_p^*$
- 2) Choose challenge $C_3 = H(\bar{y}_3)$
- 3) Compute $\bar{a}_3 = a_3 + C_3 r$
- 4) Sends $(y_3, C_3, \bar{a}_3)$ to the SN

SN then computes $\bar{y}_3 = h^{a_3} y_3^{-C_3} = h^{a_3 + C_3 r} h^{-C_3 r} = h^{a_3}$, and check if $C_3 = H(\bar{y}_3)$

The attack on YZZ+ AAKA.

1) We first show that the identity of the user (i.e., m_4) is exposed inside the proof Pres , and therefore the anonymity (unlinkability) does not hold at all. Based on the proof of π_3 , SN obtains $y_3 = -B^{-1}c_2$, which is exactly h^r due to Eq. (1). Combined with the fact that $c_2 = m_4 h^r$, SN is able to compute $m_4 = c_2/y_3$. Recall that in the YZZ+ AAKA scheme [8], m_4 is the unique field of SUPI, which is used to identify a subscriber within the HN uniquely. Therefore, SN can get the identity m_4 of UE without the secret key of LEA.

As a result, the presentation process of YZZ+ AAKA is no longer anonymous, and the multiple accesses associated with a single subscriber (UE) are linkable. That is, whenever receiving a service request, SN can record $\text{SUPI}=(pk_{HN}, m_4)$, and hence deploy a mobility tracking attack in cellular networks.

2) Now we show that SN cannot check the validity of the proof Pres , and therefore, the completeness (correctness) does not hold at all.

To prove that the user knows m_4 and a corresponding credential (signature), the user will generate a proof

$$\pi' \in \text{ZKPoK}\{(m_4, r) : \hat{\sigma}_0 \prod_{i=1}^3 \hat{\sigma}_i^{m_i} = g_1^r \hat{\sigma}_4^{-m_4}\}. \quad (2)$$

SN will check the validity of π' , as well as the equation

$$e(\hat{\sigma}, g_2) = e(\sigma', X_0). \quad (3)$$

If (2) and (3) hold, SN will pass the verification and execute the following process.

However, from Eq. (2) and (3) one cannot deduce that the user holds a valid credential (signature) for some m_4 . Recall

that the validity of a BB signature σ on message $(m_1, \dots, m_4)$ is checked via the equation

$$e(\sigma, X_0 \prod_{i=1}^4 X_i^{m_i}) = e(g_1, g_2),$$

where $X_0 = g_2^{x_0}, \dots, X_4 = g_2^{x_4}$ are public keys (of HN) and $x_0, \dots, x_4$ are corresponding secret keys. But surprisingly, the validity check equations (2) and (3) have no relation with the public keys $X_1, \dots, X_4$. To see this incompleteness clearly, consider an attack as follows. The attacker first obtains a credential $(\sigma, \sigma_0 = \sigma^{x_0}, \dots)$ on some dummy message $(\hat{m}_1, \dots, \hat{m}_4)$. Then to forge a proof on some $(m_1, \dots, m_4)$, it randomly samples r and $\hat{\sigma}_i$ for $i \in \{1, \dots, 4\}$, computes $\hat{\sigma}_0 = g_1^r \prod_{i=1}^4 \hat{\sigma}_i^{-m_i}$ and generates a zero-knowledge proof according to Eq. (2). It is easy to see that $(\hat{\sigma} = \sigma_0, \sigma' = \sigma, \hat{\sigma}_0, \dots, \hat{\sigma}_4)$ satisfies both Eq. (2) and Eq. (3), but the attacker does not hold a credential on $(m_1, \dots, m_4)$ at all.

3) Another issue is that the security of the KVAC scheme by Camenisch et al. [9] is based on the k -SCDHI assumption over a cyclic group $\mathbb{G}$, which has been proved to hold in the generic group model (GGM) [21]. To keep consistent with the AAKA scheme, Yu et al. migrated the k -SCDHI assumption to an (asymmetric) pairing group [8]. However, Yu et al. did not argue the soundness of k -SCDHI assumption over (asymmetric) pairing groups. We fix this flaw via a formal proof under GGM, see Appendix A.

To summarize, the YZZ+ AAKA scheme cannot achieve privacy (anti-tracking privacy enhancement) and completeness as it claimed, and even the description and security analysis of the scheme are fragmentary. In Section V and Appendix B, we fix all the problems by proposing two new schemes, AAKA+BB and AAKA+PS, in the enhanced model.

Key Design Pitfalls and Lessons Learned. Note that the secret key (i.e., $x_0, \dots, x_4$) of HN cannot be shared with SN. We suspect [8] attempted to transform the message authentication code (MAC, of which the verification needs secret keys) scheme KVAC in [9] into a pairing-based signature scheme to achieve public verification. To this end, they borrowed the technology of proving knowledge of a BB signature in [10]. However, their transformation failed and the scheme is not a secure signature scheme, due to the aforementioned flaw in completeness. Meanwhile, the MAC scheme in [9] is re-randomizable by masking σ with randomness r . However, such randomness cannot be reused in the (ElGamal) encryption—introduced to provide traceability—since doing so would compromise the security of the encryption itself.

When revisiting AAKA [8] and additionally introducing local revocation, we avoid this pitfall by analyzing the building blocks step by step. Taking AAKA+BB as an example, we first systematically study the transformation from MAC [9] to signatures—an aspect overlooked in YZZ+ AAKA [8]. We find that this transformation is not as straightforward as expected. Specifically, a single randomness is insufficient to hide both σ and the message, so we introduce two random values r and t to mask them separately. Moreover, the transformation relies on the k -SCDHI assumption over pairings, which was not analyzed in the original work but fixed by us in Appendix A. We refer to Section V for more details.

²The proof presented in [8] is $\text{Pres} \leftarrow (\dots, \{\hat{\sigma}_i\}_{i=0}^4, \dots)$. We believe it is a typo and present it as $\text{Pres} \leftarrow (\dots, \{\hat{\sigma}_i\}_{i=0}^4, \dots)$ here.

IV. SYNTAX AND SECURITY MODEL OF AAKA+

A. Overview of AAKA+

The architecture of an Anonymous Authentication and Key Agreement with Verifier-Local Revocation (AAKA+) scheme is illustrated in Fig. 2. The participants include the entity UE (ME+USIM/SIM/eSIM (Embedded SIM)), MNO (HN/SN), and LEA.³

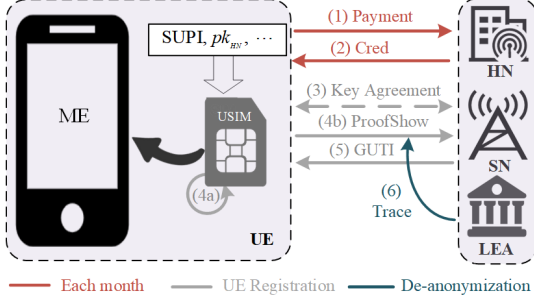


Fig. 2: Overview of AAKA+ architecture

A User Equipment (UE) refers to the user wishing to connect to the mobile network, typically including a Mobile Equipment (ME) such as a smartphone. When a user purchases a SIM card from the Home Network (HN), a Subscriber Permanent Identifier (SUPI) and the HN's identifier (the public key of the HN) is embedded into the card.

A Mobile Network Operator (MNO) refers to either the Home Network (HN) or a Serving Network (SN). (i) The HN is responsible for issuing credentials to paid subscribers according to their attributes. (ii) An SN provides local mobile communication services to UEs after verifying anonymous credential proofs.

The Law Enforcement Agency (LEA) is a lawful entity for tracing UEs from anonymous proofs when misbehaviors are detected. Furthermore, the LEA and the HN together maintain a revocation list, collecting tracing messages during the trace. Such tracing messages can be used to determine whether the associated user has been traced and revoked, whenever receiving a new proof.

We explain the workflow of Fig. 2 in a roaming scenario, i.e., a UE requests a network service via an SN without the HN's involvement. This roaming case has wider applicability compared with the non-roaming case (i.e., SN = HN).⁴

In step (1), the UE pays for the upcoming month's fee. In step (2), the HN issues a credential (a network access license) based on the attributes (m, pm) of the subscriber. Here m contains non-sensitive information including the expiration date, and pm is sampled uniformly and is associated with the real identity of the subscriber. Then the credential is privately stored in the UE. Whenever there is a need to connect to the

network, the UE starts a key agreement protocol with the SN and shares a session key in step (3), which is used to build a secure channel and guarantee subsequent communication. In step (4a), the UE generates an anonymous credential proof and sends it to the SN (step (4b)). After verifying the proof, the SN will assign the UE a Global Unique Temporary Identifier (GUTI, step (5)), allowing the UE to access the network service. When necessary, in step (6), the LEA will trace pm from an anonymous proof using their tracing secret key.

B. Syntax of AAKA+

An AAKA+ scheme consists of the following algorithms/protocols.

- $\text{para} \leftarrow \text{Setup}(1^\lambda)$: The setup algorithm takes as inputs the security parameter λ , and outputs a public parameter para . W.l.o.g., we assume para is an implicit input in the following algorithms.
- $(\text{ipk}, \text{isk}) \leftarrow \text{IKeyGen}()$: The issuer key generation algorithm (takes as inputs para and) outputs a pair of issuing keys ipk and isk .
- $(\text{tpk}, \text{tsk}) \leftarrow \text{IEAKeyGen}()$: The tracer key generation algorithm (takes as inputs para and) outputs a pair of tracing keys tpk and tsk .
- $(\text{pk}, \text{sk}) \leftarrow \text{AsymKeygen}()$: The asymmetric key generation algorithm (takes as inputs para and) outputs a pair of keys pk and sk .
- $\text{cred} \leftarrow \text{CredIssue}(\text{ipk}, \text{isk}, m, pm)$: The credential issuing algorithm takes as inputs $\text{ipk}, \text{isk}, m$ and private message pm ,⁵ and outputs a credential cred .
- $0/1 \leftarrow \text{CredVer}(\text{ipk}, m, pm, \text{cred})$: The credential verification algorithm takes as inputs $\text{ipk}, m, pm, \text{cred}$, and outputs a bit.
- $(k_{UE}, k_{MNO}) \leftarrow \langle \text{KeyEx}_{UE}(\text{pk}_{MNO}) \rightleftharpoons \text{KeyEx}_{MNO}(\text{pk}_{MNO}, \text{sk}_{MNO}) \rangle$: The key agreement protocol is executed between UE with inputs $\text{para}, \text{pk}_{MNO}$, and MNO with inputs $\text{para}, \text{pk}_{MNO}, \text{sk}_{MNO}$, and after the execution UE outputs k_{UE} , and MNO outputs k_{MNO} . Here MNO denotes either HN or SN, depending on the choice of communication models. The session keys k_{UE} and k_{MNO} are used to establish a secure channel for the subsequent communication.
- $\text{pf} \leftarrow \text{ProofShow}(\text{ipk}, \text{tpk}, m, pm, \text{cred})$: The credential showing algorithm takes as inputs $\text{ipk}, \text{tpk}, m, pm, \text{cred}$, and outputs an anonymous proof pf .
- $0/1 \leftarrow \text{ProofVer}(\text{ipk}, \text{tpk}, m, pm, \text{pf})$: The proof verification algorithm takes as inputs $\text{ipk}, \text{tpk}, m, pm, \text{pf}$, and outputs a bit.
- $\text{tm} \leftarrow \text{Trace}(\text{tsk}, m, \text{pf})$: The trace algorithm takes as inputs tsk, m, pf , and outputs a tracing message tm , which is linked to pm via some publicly evaluated function $f(\text{ipk}, pm)$.
- $0/1 \leftarrow \text{Judge}(\text{ipk}, \text{tpk}, m, \text{pf}, \text{tm})$: The judge algorithm takes as inputs $\text{ipk}, \text{tpk}, m, \text{pf}$ and a traced message tm , and outputs a bit indicating whether pf is generated by some user whose identity has been traced.

Correctness of key agreement. For any para , let $(\text{pk}, \text{sk}) \leftarrow \text{AsymKeygen}()$, and $\text{KeyEx}((\text{pk}_{MNO}) \rightleftharpoons (\text{pk}_{MNO}, \text{sk}_{MNO}))$

³The prior work by Yu et al. [8] proposed AAKA without formal definitions and security models, and there are massive ambiguous notations and definitions. In this work, we follow the main workflow of [8] and make the primitive clear by formalizing the syntax and the security properties of AAKA+ (and hence AAKA).

⁴We mainly follow AAKA's workflow in [8] that assumes a single HN. In Section VIII, we further consider decentralized HN.

⁵As mentioned above, only pm is related to a user's identity. Whenever a user shows their credential to the SN, m is public and pm is kept private.

return k_{UE} and k_{MNO} , respectively. Then with overwhelming probability, it holds that $k_{UE} = k_{MNO}$.

Correctness of authentication. For any para, message m and private message pm , let $(ipk, isk) \leftarrow \text{IKeyGen}()$, $(tpk, ts) \leftarrow \text{IEAKeyGen}()$, $cred \leftarrow \text{CredIssue}(ipk, isk, m, pm)$ and $pf \leftarrow \text{ProofShow}(ipk, tpk, m, pm, cred)$. Then with overwhelming probability, it holds that

1. $\text{CredVer}(ipk, m_1, pm, cred) = 1$.
2. $\text{ProofVer}(ipk, tpk, m, pf) = 1$.
3. $tm = f(ipk, pm)$, where $tm \leftarrow \text{Trace}(tsk, m, pf)$.
4. $\text{Judge}(ipk, tpk, m, pf', tm) = 1$, where $tm \leftarrow \text{Trace}(tsk, m, pf)$ and $pf' \leftarrow \text{ProofShow}(ipk, tpk, m, pm, cred)$.

C. Security Model

We follow the 3GPP security requirements in mobile networks [13]. At a high level, the security of AAKA+ involves two aspects: the security of key agreement, which includes the pseudorandomness of session keys, and the security of the anonymous authentication, which includes unforgeability, unlinkability, and traceability.

Pseudorandomness of session keys. The adversary cannot distinguish a real session key from a random session key.

Unforgeability. Unforgeability prevents an adversary from passing the verification of MNO without a valid credential issued by HN.

Unlinkability. Unlinkability requires that the adversary cannot tell whether two proofs pf and pf' are associated with the same user holding the same pm , if pm has not been traced.

Traceability. Traceability asks that a revoked adversary whose pm has been traced, cannot generate another proof that passes the Judge algorithm.

The pseudorandomness of session keys is the most foundational security property for (authenticated) key agreement protocols [27]–[29]. We refer to [28] for a formal security definition due to space limitation.

Before formalizing unforgeability, unlinkability, and traceability, we first define the following oracles used in the security experiments. Let (ipk, isk) be the key pair of HM (the issuer), and (tpk, ts) be the key pair of LEA (the tracer). All sets/lists are initialized to be empty.

- $\text{CRED}(m, pm)$. Compute a credential $cred \leftarrow \text{CredIssue}(ipk, isk, m, pm)$, set $\mathcal{T}[m, pm] = cred$, add (m, pm) into $\mathcal{C}$, and return $cred$.
- $\text{PROOF}(m, pm)$. Retrieve $\mathcal{T}[m, pm] = cred$ (if such a record does not exist, then the oracle invokes $\text{CRED}(m, pm)$ to create it), compute a proof $pf \leftarrow \text{ProofShow}(ipk, tpk, m, pm, cred)$, add (m, pm, pf) into $\mathcal{S}$, and return pf .
- $\text{TRACE}(m, pf)$. Return $tm \leftarrow \text{Trace}(tsk, m, pf)$ if $\text{ProofVer}(ipk, tpk, m, pf) = 1$, and $\perp$ otherwise.

Unforgeability. Consider the following unforgeability experiment. We say the AAKA+ scheme has unforgeability, if for any PPT adversary $\mathcal{A}$, it holds that

$$\text{Adv}_{\text{AAKA}+, \mathcal{A}}^{\text{unforg}}(\lambda) = \Pr[\text{Exp}_{\text{AAKA}+, \mathcal{A}}^{\text{unforg}}(\lambda) = 1] \leq \text{negl}(\lambda).$$

```

ExpAAKA+, Aunforg(λ):
para ← Setup(1λ)
(ipk, isk) ← IKeyGen(para)
(tpk, ts) ← IEAKeyGen(para)
(m*, pf*) ← ACRED(·), PROOF(·)(ipk, tpk)
If ProofVer(ipk, tpk, m*, pf*) = 1 ∧  $\nexists (m^*, \cdot) \in \mathcal{C} \wedge (m^*, \cdot, pf^*) \notin \mathcal{S}$ :
  Return 1.
Otherwise:
  Return 0.

```

```

ExpAAKA+, Aunlink(λ):
b  $\xleftarrow{\$}$  {0, 1}
para ← Setup(1λ)
(ipk, isk) ← IKeyGen(para);
(tpk, ts) ← IEAKeyGen(para)
// sample pm, pm' from the message space M randomly
pm, pm'  $\xleftarrow{\$}$  M
(m, st) ← ACRED(·), PROOF(·), TRACE(·)(ipk, tpk, isk)
cred ← CredIssue(ipk, isk, m, pm)
cred' ← CredIssue(ipk, isk, m, pm')
pf ← ProofShow(ipk, tpk, m, pm, cred)
If b = 0: pf' ← ProofShow(ipk, tpk, m, pm, cred)
If b = 1: pf' ← ProofShow(ipk, tpk, m, pm', cred')
b' ← ACRED(·), PROOF(·), TRACE(·)(st, pf, pf')
If b' = b:
  Return 1.
Otherwise:
  Return 0.

```

Unlinkability. Consider the following unlinkability experiment. An adversary $\mathcal{A}$ is admissible, if it never asks $\text{TRACE}(\cdot, pf)$ or $\text{TRACE}(\cdot, pf')$ after receiving pf and pf' . We say the AAKA+ scheme has unlinkability if for any PPT admissible adversary $\mathcal{A}$, it holds that

$$\text{Adv}_{\text{AAKA}+, \mathcal{A}}^{\text{unlink}}(\lambda) = |\Pr[\text{Exp}_{\text{AAKA}+, \mathcal{A}}^{\text{unlink}}(\lambda) = 1] - 1/2| \leq \text{negl}(\lambda).$$

```

ExpAAKA+, Atrace(λ):
para ← Setup(1λ)
(ipk, isk) ← IKeyGen(para)
(tpk, ts) ← IEAKeyGen(para)
(m, pm, st1) ← ACRED(·), PROOF(·), TRACE(·)(ipk, tpk)
cred ← CredIssue(ipk, isk, m, pm)
(pf, st2) ← ACRED(·), PROOF(·), TRACE(·)(cred, st1)
If ProofVer(ipk, tpk, m, pf) = 0:
  Return 0.
tm ← Trace(tsk, m, pf)
pf* ← ACRED(·), PROOF(·), TRACE(·)(st2)
If ProofVer(ipk, tpk, m, pf*) = 1 ∧ Judge(ipk, tpk, m, pf*, tm) = 0:
  Return 1.
Otherwise:
  Return 0.

```

Traceability. Consider the following traceability experiment. An adversary $\mathcal{A}$ is admissible, if $(m, \cdot) \notin \mathcal{C}$, and $(m, \cdot, pf^*) \notin \mathcal{S}$. We say the AAKA+ scheme has traceability if for any PPT admissible adversary $\mathcal{A}$, it holds that

$$\text{Adv}_{\text{AAKA}+, \mathcal{A}}^{\text{trace}}(\lambda) = \Pr[\text{Exp}_{\text{AAKA}+, \mathcal{A}}^{\text{trace}}(\lambda) = 1] \leq \text{negl}(\lambda).$$

V. CONSTRUCTION OF AAKA+BB

In this section, we propose our first scheme AAKA+BB, based on the variant Boneh-Boyen (BB) signature scheme [9], [11]. Our second scheme AAKA+PS based on the Pointcheval-Sanders (PS) signature scheme [12] is deferred to Appendix B due to the space limitation. The scheme comprises five blocks: System Setup, Credential Issuance, Key Agreement, Credential Showing and Verification, and Trace.

(1) System Setup

We assume all parties in AAKA+ have access to global parameters, and in particular, the public parameters are securely provisioned into the root-of-trust SIM/USIM. We refer to TR 33.935 [30] and [8] for more non-technical details about the initiation of parameters and keys.

• **Setup** : $\text{para} \leftarrow \text{Setup}(1^\lambda)$.

Generate a type-3 pairing group $\Phi = (q, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, g_1, g_2, e)$, where g_1, g_2 are the generators of $\mathbb{G}_1$ and $\mathbb{G}_2$, respectively. Let $H_0 : \{0, 1\}^* \rightarrow \mathbb{Z}_q^*$, $H_1 : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$, $H_2 : \{0, 1\}^* \rightarrow \mathbb{G}_1^*$, $H_{zk} : \{0, 1\}^* \rightarrow \mathbb{Z}_q^*$ be hash functions (which work as random oracles in this paper). Return the system parameter $\text{para} = (\Phi, H_0, H_1, H_2, H_{zk})$.

• **IKeyGen** : $(\text{ipk}, \text{isk}) \leftarrow \text{IKeyGen}(\text{para})$.

The HN executes this to generate a pair of keys to issue credentials. It randomly chooses $x_0, x_1, x_2 \in \mathbb{Z}_q^*$ as isk , and computes $\text{ipk} = (X_0, X_1, X_2) = (g_2^{x_0}, g_2^{x_1}, g_2^{x_2})$.

• **LEAKeyGen** : $(\text{tpk}, \text{tsk}) \leftarrow \text{LEAKeyGen}(\text{para})$.

The LEA executes this to generate a pair of keys for tracing. It randomly chooses $z \in \mathbb{Z}_q^*$ as tsk , and computes $Z = g_2^z$ as tpk .

• **AsymKeyGen** : $(\text{pk}, \text{sk}) \leftarrow \text{AsymKeygen}(\text{para})$.

The HN/SN executes this to generate a pair of keys for the key agreement protocol. It randomly chooses $y \in \mathbb{Z}_q^*$ as sk , and computes $Y = g_1^y$ as pk .

(2) Credential Issuance

The HN executes the credential issuance for paid subscribers. There are 2 messages/attributes m and pm embedded in a credential cred , where $m \in \mathbb{Z}_q$ represents the public information, and $pm \in \mathbb{Z}_q$ is a private message randomly sampled when registration. Furthermore, the HN holds a private mapping between the user's unique identity and pm (so that LEA can locate the user after tracing pm).

• **Credential Issue**: $\text{cred} \leftarrow \text{CredIssue}(\text{ipk}, \text{isk}, m, pm)$.

Let $\text{isk} = (x_0, x_1, x_2)$. Given m and pm , HN calls the signing algorithm to obtain a signature $\sigma = g_1^{\frac{x_0 + x_1 m + x_2 pm}{x_0 + x_1 m + x_2 pm}}$. Furthermore, HN computes $\sigma_0 = \sigma^{x_0}$, $\sigma_1 = \sigma^{x_1}$, and $\sigma_2 = \sigma^{x_2}$, and generates a zero-knowledge proof

$$\pi_0 \in \text{ZKPoK}\{(x_0, x_1, x_2) : \bigwedge_{i=0}^2 \sigma_i = \sigma^{x_i} \wedge X_i = g_2^{x_i}\}.$$

The construction of π_0 is shown in Appendix C. Finally, HN sends (m, pm) and $\text{cred} = (\sigma, \sigma_0, \sigma_1, \sigma_2)$ to USIM. Note that HN can only provide the subscription credentials to USIM via the 5G OTA remote provisioning channel. Therefore, (m, pm, cred) can be securely sent to USIM.

• **Credential Verification**: $0/1 \leftarrow \text{CredVer}(\text{ipk}, m, pm, \text{cred})$.

UE executes this to check the validity of cred . It parses $\text{cred} = (\sigma, \sigma_0, \sigma_1, \sigma_2)$, and returns 1 if both the following two checks pass:

1) $\sigma_0 \cdot \sigma_1^m \cdot \sigma_2^{pm} = g_1$, and 2) π_0 is valid w.r.t. instance $(\text{ipk}, \sigma, \sigma_0, \sigma_1, \sigma_2)$.

Then the UE accepts cred and stores it on USIM.

(3) Key Agreement

Whenever a UE starts a session with an MNO (SN or HN), they first execute an (authenticated) key agreement protocol and agree on a session key k_s , which is used to establish a secure channel for the following credential showing and verification. The key agreement sub-protocol is a variant of the HMQV protocol [29] that only the MNO side holds a long-term secret key.

1. UE retrieves MNO's public key Y .

2. UE randomly samples $a \in \mathbb{Z}_q^*$, and sends $A = g_1^a$ to MNO.

3. MNO randomly samples $b \in \mathbb{Z}_q^*$, computes $B = g_1^b$, $\delta = H_0(Y, A, B) \in \mathbb{Z}_q^*$, and $K = (A)^{b+\delta y}$. Let $\tau = H_1(K, 0)$ and $k_s = H_1(K, 1)$. Then MNO sends (B, τ) to UE and sets k_s as the session key.

4. After receiving (B, τ) from MNO, the UE computes $\delta = H_0(Y, A, B)$ and $K = (B \cdot Y^\delta)^a$. Then it checks whether $\tau = H_1(K, 0)$. If yes, UE sets $k_s = H_1(K, 1)$ as the session key.

All following communications are encrypted by the session key k_s in an authenticated encryption mode. For simplicity, we omit the encryption by k_s in the next subsection but just focus on the credential showing and verification.⁶

(4) Credential Showing and Verification

The Credential Showing and Verification sub-protocol allows the UE to generate a non-interactive proof showing the ownership of a credential to the MNO.

• **ProofShow** : $\sigma \leftarrow \text{ProofShow}(\text{ipk}, \text{tpk}, m, pm, \text{cred})$.

Let $\text{cred} = (\sigma, \sigma_0, \sigma_1, \sigma_2)$. UE randomly samples $r, t, u \in \mathbb{Z}_q^*$, and computes

- $\hat{\sigma} = \sigma^r \in \mathbb{G}_1$,
- $C_1 = X_0 X_1^m X_2^{pm} g_2^t \in \mathbb{G}_2$,
- $C_2 = g_1^r \hat{\sigma}^t \in \mathbb{G}_1$,
- $C_3 = g_2^u \in \mathbb{G}_2$,
- $C_4 = Z^u \cdot g_2^{pm} \in \mathbb{G}_2$,
- $C_5 = h^{pm} \in \mathbb{G}_1$, where $h = H_2(\hat{\sigma}, C_1, C_2, C_3, C_4, m) \in \mathbb{G}_1$.

Meanwhile, UE generates a zero-knowledge proof

$$\pi_1 \in \text{ZKPoK}\{(pm, t, r, u) : C_1 = X_0 X_1^m X_2^{pm} g_2^t \wedge C_2 = g_1^r \hat{\sigma}^t \wedge C_3 = g_2^u \wedge C_4 = Z^u \cdot g_2^{pm} \wedge C_5 = h^{pm}\}.$$

UE sets $\text{pf} = (\hat{\sigma}, C_1, C_2, C_3, C_4, C_5, \pi_1)$. Finally, it sends (m, pf) to MNO. (Actually, (m, pf) is sent after encryption using the session key k_s .)

The construction of π_1 is shown in Appendix C. Note that we take the Fiat-Shamir paradigm [25] to design the zero-knowledge scheme and the transcript (A, B, τ) of the key agreement sub-protocol is included in the input of the hash function H_{zk} (which works as a random oracle), to prevent

⁶The key agreement phase seems to be independent of the main body of AAKA+. Here we follow [8] to include the key agreement phase as a part of the whole scheme and formally describe the syntax and the security definition.

the UE from reusing a credential proof to an MNO (non-transferability).

- **ProofVer** : $0/1 \leftarrow \text{ProofVer}(\text{ipk}, \text{tpk}, m, \text{pf})$.

Upon receiving $(m, \text{pf} = (\hat{\sigma}, C_1, C_2, C_3, C_4, C_5, \pi_1))$, SN executes as follows.

1. Check the validity of UE's current subscription based on the information of m . If the subscription has expired, SN rejects the registration request.

2. Identify the HN's public key and retrieve the relevant $\text{ipk} = (X_0, X_1, X_2)$.

3. Check whether $e(\hat{\sigma}, C_1) = e(C_2, g_2)$.

4. Check the validity of π_1 w.r.t. instance $(\text{ipk}, \text{tpk}, \hat{\sigma}, C_1, C_2, C_3, C_4, C_5, m)$. The detail of the zero-knowledge scheme is shown in Appendix C.

Note. To accelerate verification, for proofs submitted by multiple users, SN can conduct aggregated verification to reduce communication delay and verification time. For $j = 0, \dots, n$, SN executes the aggregated verification for $\prod_{j=0}^n e((\hat{\sigma})_j, (C_1)_j) = e(\prod_{j=0}^n (C_2)_j, g_2)$, which can compress the pairing operation from $2n$ to $n + 1$.

If all the verification succeeds, then the corresponding Access and Mobility Management Function (AMF, which cooperates with SN and is responsible for handling signaling and connection management for mobile devices) will send a temporary identifier (i.e., GUTI) to the UE to finish this registration.

(5) Trace

- **Trace** : $\text{tm} \leftarrow \text{Trace}(\text{tsk}, m, \text{pf})$.

Whenever a suspicious request or illegal service is monitored, the LEA will trace an anonymous proof $\text{pf} = (\hat{\sigma}, C_1, C_2, C_3, C_4, C_5, \pi_1)$ using its secret key $\text{isk} = z$. The LEA computes

$$\text{tm} = C_4/C_3^z,$$

which is exactly g_2^{pm} if pf is well-formed.

The LEA and the HN together maintain a revocation list $\mathcal{RL}$ collecting all traced g_2^{pm} . Note that g_2^{pm} is determined by pm , the private message/attribute related to the subscriber's unique identity.

- **Judge** : $0/1 \leftarrow \text{Judge}(\text{ipk}, \text{tpk}, m, \text{pf}', \text{tm})$.

Whenever receiving a new proof $\text{pf} = (\hat{\sigma}, C_1, C_2, C_3, C_4, C_5, \pi_1)$ and the corresponding message m , SN checks whether there exists $W \in \mathcal{RL}$ s.t.

$$e(h, W) = e(C_5, g_2).$$

Note. We include the transcript (A, B, τ) of the key agreement sub-protocol into the challenge of ZKPoK, to ensure that the proof is uniquely used in this session. All the necessary information, including m and the proof pf , are packaged and sent to the SN via a secure channel established by the session key k_s .

VI. SECURITY ANALYSIS

Theorem 1. *If the SXDH assumption holds in $\mathbb{G}_1$ and $\mathbb{G}_2$, the BB signature scheme is unforgeable, and π_1 is a proof of knowledge scheme, then AAKA+BB is a secure AAKA+ scheme.*

Proof. **1) Unforgeability.** Suppose that with non-negligible probability the adversary $\mathcal{A}$ finally outputs $(m, \text{pf} = (\hat{\sigma}, C_1, C_2, C_3, C_4, C_5, \pi_1))$ in the unforgeability experiment $\text{Exp}_{\text{AAKA+BB}, \mathcal{A}}^{\text{unforg}}(\lambda)$, and $\text{ProofVer}(\text{ipk}, \text{tpk}, m, \text{pf}) = 1$. Let $h = H_2(\hat{\sigma}, C_1, \dots, C_4, m)$. Recall that π_1 is a zero-knowledge proof for the witness (pm, t, r, u) such that

$$\pi_1 \in \text{ZKPoK}\{(pm, t, r, u) : C_1 = X_0 X_1^m X_2^{pm} g_2^t \wedge C_2 = g_1^r \hat{\sigma}^t \wedge C_3 = g_2^u \wedge C_4 = Z^u \cdot g_2^{pm} \wedge C_5 = h^{pm}\}.$$

Thanks to the extractability of ZKPoK, with non-negligible probability we can extract a witness (pm, t, r, u) that satisfies the above equations.⁷ Therefore, $\sigma = \hat{\sigma}^{r^{-1}}$ is a valid BB signature satisfying $e(\sigma, X_0 X_1^m X_2^{pm}) = e(g_1, g_2)$.

We further analyze in two cases:

Case 1. $\mathcal{A}$ never asks $\text{CRED}(m, pm)$ or $\text{PROOF}(m, pm)$. Therefore, σ is a valid forgery on a fresh message (m, pm) , and we can construct a reduction algorithm that breaks the unforgeability of the underlying BB signature scheme.

Case 2. $\mathcal{A}$ asks a query $\text{PROOF}(m, pm)$, and there is a record $(m, pm, \text{pf}' \neq \text{pf}) \in \mathcal{S}$. In this case, we design a simulator that simulates an anonymous proof pf' without knowing a BB signature σ on (m, pm) , so that we can construct a reduction algorithm to break the unforgeability of the BB signature scheme.

The simulator works as follows. Given m and pm , it randomly samples $\alpha, \beta, u \in \mathbb{Z}_q$, and computes $\hat{\sigma} = g_1^\alpha$, $C_1 = g_2^\beta$, and $C_2 = g_1^{\alpha\beta}$. Then, it computes C_3, C_4, C_5 as normal, and generates a proof π_1 using the zero-knowledge simulator of ZKPoK.

It is easy to see that $\hat{\sigma}$ and C_1 are uniformly distributed as those in ProofShow, and C_2 is uniquely determined by $\hat{\sigma}$ and C_1 due to the verification equation $e(\hat{\sigma}, C_1) = e(C_2, g_2)$. Meanwhile, C_2, C_3, C_4 are generated as normal, and π' distributes identically as an honestly generated one due to the zero-knowledge property of ZKPoK. Therefore, the simulator can answer $\text{PROOF}(m, pm)$ perfectly. Taking all together, the unforgeability of AAKA+BB holds.

2) Unlinkability. As shown above, due to the randomness of r and t , the term $(\hat{\sigma}, C_1, C_2)$ in a proof pf does not leak any information of pm . Therefore, let us focus on the leakage (of pm) from (C_3, C_4, C_5) .

We start from the case $b = 0$ in the unlinkability experiment $\text{Exp}_{\text{AAKA+BB}, \mathcal{A}}^{\text{unlink}}$. That is, $\mathcal{A}$ receives two proofs $\text{pf} = (\hat{\sigma}, C_1, \dots, C_5, \pi_1)$ and $\text{pf}' = (\hat{\sigma}', C_1', \dots, C_5', \pi_1')$, which are generated from the same cred for some random pm .

⁷Actually the extracted witness might be different from the witness (pm, t, r, u) used in generating π_1 , but this does not affect the following proofs. For the simplicity of notations, we assume the extracted witness is exactly (pm, t, r, u) .

Recall that $C_5 = h^{pm}$ and $C'_5 = (h')^{pm}$ for $h = H_2(\hat{\sigma}, C_1, \dots, C_4, m)$ and $h' = H_2(\hat{\sigma}', C'_1, \dots, C'_4, m)$. Let u and u' be the randomness used to generate $C_3 = g_2^u$ and $C'_3 = g_2^{u'}$, and $h = g_1^\omega$, $h' = g_1^{\omega'}$ for random $\omega, \omega' \in \mathbb{Z}_q^*$. Let $\text{tpk} = Z = g_2^z$. In the case $b = 0$, the information $\mathcal{A}$ obtains from tpk , pf , and pf' is in the form of

$$\mathcal{D}_1 = \begin{pmatrix} g_2 & g_2^z & g_2^u & g_2^{uz+pm} & g_1 & g_1^\omega & g_1^{\omega \cdot pm} \\ g_2 & g_2^z & g_2^{u'} & g_2^{u'z+pm} & g_1 & g_1^{\omega'} & g_1^{\omega' \cdot pm} \end{pmatrix}.$$

We prove that the following distributions are all indistinguishable.

$$\begin{aligned} \mathcal{D}_1 &\approx \begin{pmatrix} g_2 & g_2^z & g_2^u & g_2^v & g_1 & g_1^\omega & g_1^{\omega \cdot pm} \\ g_2 & g_2^z & g_2^{u'} & g_2^{v'} & g_1 & g_1^{\omega'} & g_1^{\omega' \cdot pm} \end{pmatrix}, v, v' \leftarrow \mathbb{Z}_q \\ &\quad // \text{DDH assum. in } \mathbb{G}_2 \\ &\approx \begin{pmatrix} g_2 & g_2^z & g_2^u & g_2^v & g_1 & g_1^\omega & g_1^{\omega \cdot pm} \\ g_2 & g_2^z & g_2^{u'} & g_2^{v'} & g_1 & g_1^{\omega'} & g_1^{\gamma} \end{pmatrix}, \gamma \leftarrow \mathbb{Z}_q \\ &\quad // \text{DDH assum. in } \mathbb{G}_1 \\ &\equiv \begin{pmatrix} g_2 & g_2^z & g_2^u & g_2^v & g_1 & g_1^\omega & g_1^{\omega \cdot pm} \\ g_2 & g_2^z & g_2^{u'} & g_2^{v'} & g_1 & g_1^{\omega'} & g_1^{\omega' \cdot pm'} \end{pmatrix} \\ &\quad // pm' \text{ is randomly sampled} \\ &\approx \begin{pmatrix} g_2 & g_2^z & g_2^u & g_2^{uz+pm} & g_1 & g_1^\omega & g_1^{\omega \cdot pm} \\ g_2 & g_2^z & g_2^{u'} & g_2^{u'z+pm'} & g_1 & g_1^{\omega'} & g_1^{\omega' \cdot pm'} \end{pmatrix} \\ &\quad // \text{DDH assum. in } \mathbb{G}_2. \end{aligned}$$

The last distribution is exactly the case $b = 1$ in the unlinkability experiment, which concludes the proof.

3) Traceability. In the experiment of traceability, $\mathcal{A}$ first obtains a credential cred on (m, pm) and generates a valid proof pf . Then pf is traced and the tracing message tm is exposed, and $\mathcal{A}$ is asked to output another proof pf^* .

Due to the unforgeability, pf must contain a ciphertext $(C_3, C_4) = (g_2^u, Z^u \cdot g_2^{pm})$, as otherwise $\mathcal{A}$ breaks the unforgeability. Therefore, $\text{Trace}(\text{tsk}, m, \text{pf})$ outputs $\text{tm} = g_2^{pm}$. Let $\text{pf}^* = (\hat{\sigma}^*, C_1^*, \dots, C_5^*, \pi_1^*)$ and $h = H_2(\hat{\sigma}^*, C_1^*, \dots, C_4^*, m)$. To make $\text{Judge}(\text{ipk}, \text{tpk}, m, \text{pf}^*, \text{tm}) = 0$, it must hold that

$$e(C_5^*, g_2) \neq e(h, g_2^{pm}).$$

That is, there exists some $pm' \neq pm$ such that $C_5^* = g_2^{pm'} \neq g_2^{pm}$. Due to the extractability of ZKPoK, we can extract a witness (pm', t, r, u) as well as a signature σ' such that, σ' is a valid BB signature on message (m, pm') , which breaks the unforgeability of the underlying BB signature scheme.

4) Pseudorandomness of session keys. The key agreement sub-protocol is derived from the HMQV scheme, and we refer to [29] for the proof. $\square$

VII. THE EVALUATION OF AAKA+

In this section, we implement AAKA+BB (Section V) and AAKA+PS (Appendix B), and evaluate their efficiency. The program is written in Python and the code is publicly available.⁸

We provide a prototype system with the following experimental environment:

1) MNO (i.e., HN/SN) and LEA are implemented on a standard server (Intel Core Xeon(R) Platinum 8369HB CPU at 3.30GHz, running Ubuntu 20.04.6).

2) UE is implemented on a resource-constrained device. Due to proprietary and security restrictions, modifying the standard cell (e)SIM without carrier authorization prevents mobile network connection. To simulate a variety of cellular-connected devices, we emulate the Mobile Equipment (ME) with a Raspberry Pi 4 Model B Rev 1.5, as its 1.8GHz 64-bit quad-core Cortex A72 processor and 4GB RAM (32GB SD card) are similar to mid-range Android phones.⁹

A. Implementation

Our implementation uses standard cryptographic parameters, including Curve25519 in Montgomery form and ANSI-X9.63 KDF, as recommended in 3GPP 33.501 [13] for Subscription Concealed Identifier (SUCI) (de)concealment processes during the Cred Issuance phase in both 5G-AKA [13] and our schemes. The ECIES symmetric encryption algorithm employs AES-128, and the ANSI-X9.63 KDF is implemented with a 256-bit key length using the *cryptography* library¹⁰ in Python. All hash functions are implemented with SHA-256.

We utilize the efficient pairing-friendly BN (Barreto-Naehrig) curve, BN-256, which provides approximately 100-bit security due to recent attacks [31]. The BN-256 testing is carried out using *bplib*,¹¹ a Python library implementing support for computations on pairing groups.

1) Timing benchmark: Table II illustrates the time cost for executing each procedure in AAKA+. Each value represents the average time, measured in milliseconds, over 1,000 runs.

TABLE II: Time costs (in milliseconds) for different stages in AAKA+BB and AAKA+PS, with pre-computation times indicated in parentheses. “SN/HN” denotes either SN (roaming scenarios) or HN (non-roaming scenarios), and the notation “~” denotes approximate time.

Different stages	Entity	AAKA+BB	AAKA+PS
CredIssue	HN	6.82	3.98
CredVer	UE	38.82	12.17
Key Agreement	UE	~4.30	~4.30
Key Agreement	SN/HN	~0.89	~0.89
ProofShow	UE	18.40 (+29.45)	18.09(+26.10)
ProofVer	SN/HN	19.93	18.87
Trace	LEA	~0.96	~0.96

In our AAKA+BB scheme, credential issuance on the HN (run on the server) takes 6.82 ms, while verifying the credential on the UE (run on Raspberry Pi) takes 38.82 ms. Fortunately, this process typically occurs only once a month. After verification, the ME stores the credential as a binary EF file on the USIM. Additionally, the AAKA+PS scheme is significantly more efficient in this process. The computation of credential issuance on the HN takes only 3.98 ms, and verifying it on the

⁹Qualcomm Snapdragon 620/652 (Android-based) devices feature a 4x ARM Cortex-A72 MPcore CPU.

¹⁰<https://github.com/pyca/cryptography/>

¹¹<https://pypi.org/project/bplib/>

⁸https://github.com/XieMin-snnu/AAKA_Plus

ME takes 12.17 ms. For the key agreement phase of AKA+, the UE generates the private key and computes the shared key within the SIM card using the ME and on-card cipher suites, and the SN executes on the server simultaneously. This key agreement procedure is the same in AKA+BB and AKA+PS, with a deviation in our implementation ≤ 0.03 ms. It takes 4.30 ms for the UE and 0.89 ms for the SN.

The ME on the UE performs ProofShow for anonymous credential presentations. Generating a proof on the UE takes 18.40 ms in AKA+BB, with additional pre-computation for $cmt_1, \dots, cmt_5$ for π_1 taking 29.45 ms. The pre-computation significantly shortens the waiting time (of generating proof) and improves users' experience. Similarly, in AKA+PS, this process takes 18.09 ms, with pre-computation for $cmt_1, \dots, cmt_4$ for π_3 taking 26.10 ms. The verification operation on the SN takes 19.93 ms in AKA+BB (18.87 ms in AKA+PS).

The tracing process, executed by the LEA, requires only 1 group exponentiation on $\mathbb{G}_2$ and 1 multiplication on $\mathbb{G}_2$, and it takes ~ 0.96 ms in both AKA+BB and AKA+PS.

2) Communication complexity: Table III shows the communication complexity for each stage in AKA+. Using the BN-256 curve parameters, we allocate 32 bytes for an element in $\mathbb{Z}_q$, 33 bytes for $\mathbb{G}_1$, 65 bytes for $\mathbb{G}_2$, and 384 bytes for $\mathbb{G}_T$. Additionally, 32 bytes are used to represent a message or attribute (m or pm). A credential is 260 bytes in AKA+BB and 194 bytes in AKA+PS. The storage of the cred (by USIM) is 97 bytes ($1[\mathbb{G}_1] + 2[\mathbb{Z}_q]$) in AKA+BB and 130 bytes ($2[\mathbb{G}_1] + 2[\mathbb{Z}_q]$) in AKA+PS. The size of an anonymous proof, which embeds the private message/attribute, is approximately 0.5 KB.

TABLE III: Communication complexity of AKA+ (based on the BN-256 curve). The notations $[\mathbb{Z}_q]$, $[\mathbb{G}_1]$, and $[\mathbb{G}_2]$ denote the sizes of elements in $\mathbb{Z}_q$, $\mathbb{G}_1$ and $\mathbb{G}_2$, respectively. λ denotes the security parameter.

Components	Size	
	AKA+BB	AKA+PS
CredIssue	$4([\mathbb{G}_1] + [\mathbb{Z}_q])$	$2[\mathbb{G}_1] + 4[\mathbb{Z}_q]$
Key Agreement (UE)	$1[\mathbb{G}_1]$	$1[\mathbb{G}_1]$
Key Agreement (SN/HN)	$1[\mathbb{G}_1] + \lambda$	$1[\mathbb{G}_1] + \lambda$
ProofShow	$3([\mathbb{G}_1] + [\mathbb{G}_2]) + 5[\mathbb{Z}_q]$	$3([\mathbb{G}_1] + [\mathbb{G}_2]) + 4[\mathbb{Z}_q]$
Trace	$1[\mathbb{G}_2]$	$1[\mathbb{G}_2]$

B. Comparisons

To demonstrate the practicality of our solution, we test our benchmark against the 5G-AKA protocol [13], implemented in the same hardware environment and simulating communication among different devices through different ports. We compare the total time consumption with the current UE authentication in 5G-AKA. To eliminate unnecessary variables, we ignore transmission delays among different devices in the network (e.g., SEAF, AUSF). Since the SIM cannot perform ECIES, we used the ME to calculate SUCI as an alternative method, allowed under 3GPP TS 31.102 Section 5.3.47 [32].

On the UE side of 5G-AKA, the UE's computation involves encrypting SUCI with the HN's public key using ECIES, verifying the authenticity and integrity of RAND after receiving the response (RAND, AUTN) from HN, and calculating the

key material for subsequent procedures (i.e., K_{seaf}) to validate the response. According to [8], these processes involve prefixed on-card KDF functions f_1, f_2, f_5 , lightweight XOR and MAC set by MNOs (step 7, Fig. 5 in [8]), and generating RES* to validate the response (step 8, Fig. 5 in [8]). Based on the open-source library,¹² we implement the aforementioned ECIES functions through the standard cryptography library *cryptography*,¹³ including $f_1, \dots, f_5$, x9.63 KDFS, EC-DH, SHA256-HMAC, and AES-128 with CTR mode. Note that the SIM card cannot deploy the mentioned ECIES. Therefore, we use the ME to complete the UE-side computation in 5G-AKA, which takes 1.62 ms. Notably, SUCI concealment is the most time-consuming operation, taking 1.20 ms and accounting for 74% of the total time. On the MNO side, the calculation is mainly handled by the HN, and the SN acts as the communication node (the communication delay is determined by the network). Thus, we mainly focus on the time consumption at the HN for the MNO side, which is 0.87ms.

5G-AKA employs symmetric cryptographic primitives, where the challenge-response process necessitates the HN involvement in every UE registration, posing impracticality in roaming scenarios. In contrast, AKA+ allows the SN to independently authenticate unknown UEs without interacting with the HN, and it takes just ~ 18 ms to generate an anonymous proof for a constrained device.

VIII. FURTHER EXTENSIONS AND DISCUSSIONS

Extension of the Message Space: The construction of AKA+BB (resp., AKA+PS) relies on the Boneh-Boyen [11] [9] (resp., the Pointcheval-Sanders [12]) multi-message signature scheme. In this work, we separate the attributes of a user into two parts, public message m and private message pm . It is easy to extend our constructions to support more attributes ($m_1, \dots, m_k, pm_1, \dots, pm_{k'}$) for any $k, k' \geq 1$, by increasing the dimension of the secret key isk .

Decentralized HN (The Issuer): In this work, we consider the issue center HN as a single entity, and HN is responsible for selecting a random pseudorandom identity (pm) and issuing a credential/signature for a user. To improve the reliability and trust among users, it is natural to decentralize HN to a sequence of entities using threshold signatures and secret sharing schemes. Taking Shamir's polynomial secret sharing scheme [33], we can extend AKA+PS to support decentralized issue centers. The threshold Boneh-Boyen signature scheme can be found in [34].

Acceleration from Symmetric Methods for Identification: Both AKA+BB and AKA+PS involve an (authenticated) key agreement sub-protocol to create a session key to guarantee further communication between UE and HN/SN. If at the stage of (2) Cred (see Fig. 2) the UE and the HN pre-share a secret key k , and later the UE works in the non-roaming mode (SN = HN), then we can use symmetric methods (like MAC) to generate a session key, just like 5G-AKA [13], avoiding the computation/communication cost of executing asymmetric cryptographic algorithms.

¹²<https://github.com/LafayetteTheLancelot/AKA5G>

¹³<https://github.com/pyca/cryptography/>

IX. CONCLUSION

In this work, we identify several fatal flaws in the YZZ+ AAKA scheme, demonstrating that AAKA cannot achieve the claimed privacy and correctness. Following the design goals of AAKA, we propose AAKA+ to ensure privacy against mobile tracking. In addition to the privacy and traceability features introduced in [8], AAKA+ enables the MNO to immediately determine whether the associated subscriber has been traced and revoked upon receiving an anonymous proof. We formally define the syntax and security model of AAKA+ and propose two concrete schemes, AAKA+BB and AAKA+PS, based on the Boneh-Boyen and Pointcheval-Sanders signature schemes, respectively. Both schemes are pairing-free on the user equipment side, compatible with existing cellular infrastructure, and exhibit high efficiency, as confirmed by the experimental results.

In future work, we aim to make the schemes more compact and introduce more symmetric techniques, such as message authentication codes, to further improve the efficiency. Another direction we are exploring is the design of lattice-based AAKA+ schemes to provide post-quantum security.

Acknowledgements

Haibin Zhang was supported by the National Key R&D Program of China (2022YFB2701500). Yong Yu was supported by the National Cryptologic Science Fund of China (2025NCSF02025), the National Natural Science Foundation of China (U24B20149, 62272385, U23A20302, 62311540156), the Key Research and Development Program of Shaanxi (2024GX-ZDCYL-01-09), and the Major Program of Shandong Provincial Natural Science Foundation for the Fundamental Research (ZR2022ZD03). Yanqi Zhao was supported by the National Natural Science Foundation of China (62202375) and the Youth Innovation Team of Shaanxi Universities (24JP180). Baodong Qin was supported by the National Natural Science Foundation of China (62372370). Jianting Ning was supported by the National Natural Science Foundation of China (62425205, 12441101). Xiaoyi Yang was supported by the Key Research and Development Program of Shaanxi (2024GX-ZDCYL-01-15). Xiangyu Liu is the corresponding author.

REFERENCES

- [1] R. Borgaonkar, L. Hirschi, S. Park, and A. Shaik, "New privacy threat on 3g, 4g, and upcoming 5g AKA protocols," *Proc. Priv. Enhancing Technol.*, vol. 2019, no. 3, pp. 108–127, 2019.
- [2] M. Rath, "Rethinking reverse location search warrants," *Journal of Criminal Law and Criminology*, 2021.
- [3] "3GPP TS 33.107 V17.0.0. Lawful Interception (LI) architecture and functions," 2022.
- [4] "3GPP TS 33.126 V16.2.0. Lawful Interception requirements," 2020.
- [5] C. Héban and D. Pointcheval, "Traceable constant-size multi-authority credentials," in *SCN 2022*, C. Galdi and S. Jarecki, Eds., vol. 13409. Springer, 2022, pp. 411–434.
- [6] J. Camenisch and A. Lysyanskaya, "An efficient system for non-transferable anonymous credentials with optional anonymity revocation," in *Advances in Cryptology - EUROCRYPT 2001*, vol. 2045, 2001, pp. 93–118.
- [7] D. Boneh and H. Shacham, "Group signatures with verifier-local revocation," in *CCS 2004*. ACM, 2004, pp. 168–177.
- [8] H. Yu, C. Du, Y. Xiao, A. Keromytis, C. Wang, R. Gazda, Y. T. Hou, and W. Lou, "Aaka: An anti-tracking cellular authentication scheme leveraging anonymous credentials," in *NDSS 2024*, 2024.
- [9] J. Camenisch, M. Drijvers, P. Dzurenda, and J. Hajny, "Fast keyed-verification anonymous credentials on standard smart cards," in *SEC 2019*, vol. 562. Springer, 2019, pp. 286–298.
- [10] J. Camenisch, M. Drijvers, and J. Hajny, "Scalable revocation scheme for anonymous credentials based on n-times unlinkable proofs," in *WPES@CCS 2016*. ACM, 2016, pp. 123–133.
- [11] D. Boneh and X. Boyen, "Short signatures without random oracles," in *Advances in Cryptology - EUROCRYPT 2004*, vol. 3027. Springer, 2004, pp. 56–73.
- [12] D. Pointcheval and O. Sanders, "Short randomizable signatures," in *Topics in Cryptology - CT-RSA 2016*, vol. 9610. Springer, 2016, pp. 111–126.
- [13] "3GPP TS 33.501 V16.10.0: Security architecture and procedures for 5G System," 2022.
- [14] A. Koutsos, "The 5g-aka authentication protocol privacy," in *IEEE European Symposium on Security and Privacy, EuroS&P 2019, Stockholm, Sweden, June 17-19, 2019*. IEEE, 2019, pp. 464–479.
- [15] Y. Wang, Z. Zhang, and Y. Xie, "Privacy-preserving and standard-compatible AKA protocol for 5g," in *USENIX Security 2021*, M. D. Bailey and R. Greenstadt, Eds. USENIX Association, 2021, pp. 3595–3612.
- [16] B. Hong, S. Bae, and Y. Kim, "GUTI reallocation demystified: Cellular location tracking with changing temporary identifier," in *NDSS 2018*, 2018.
- [17] C. Du, H. Yu, Y. Xiao, W. Lou, C. Wang, R. Gazda, and Y. T. Hou, "Mobile tracking in 5g and beyond networks: Problems, challenges, and new directions," in *MASS 2022*. IEEE, 2022, p. 426–434.
- [18] P. Schmitt and B. Raghavan, "Pretty good phone privacy," in *USENIX Security 2021*, 2021, pp. 1737–1754.
- [19] S. D. Galbraith, K. G. Paterson, and N. P. Smart, "Pairings for cryptographers," *Discret. Appl. Math.*, vol. 156, no. 16, pp. 3113–3121, 2008.
- [20] J. Groth and A. Sahai, "Efficient non-interactive proof systems for bilinear groups," in *Advances in Cryptology - EUROCRYPT 2008*, vol. 4965, 2008, pp. 415–432.
- [21] V. Shoup, "Lower bounds for discrete logarithms and related problems," in *Advances in Cryptology - EUROCRYPT '97*, vol. 1233. Springer, 1997, pp. 256–266.
- [22] M. Bellare and O. Goldreich, "On defining proofs of knowledge," in *Advances in Cryptology - CRYPTO '92*, vol. 740. Springer, 1992, pp. 390–420.
- [23] C. Hazay and Y. Lindell, "A note on zero-knowledge proofs of knowledge and the ZKPOK ideal functionality," *IACR Cryptol. ePrint Arch.*, p. 552, 2010.
- [24] C. Schnorr, "Efficient identification and signatures for smart cards (abstract)," in *Advances in Cryptology - EUROCRYPT '89*, ser. Lecture Notes in Computer Science, vol. 434. Springer, 1989, pp. 688–689.
- [25] A. Fiat and A. Shamir, "How to prove yourself: Practical solutions to identification and signature problems," in *Advances in Cryptology - CRYPTO '86*, vol. 263. Springer, 1986, pp. 186–194.
- [26] J. Camenisch and M. Stadler, "Efficient group signature schemes for large groups (extended abstract)," in *Advances in Cryptology - CRYPTO '97*, vol. 1294. Springer, 1997, pp. 410–424.
- [27] M. Bellare and P. Rogaway, "Entity authentication and key distribution," in *Advances in Cryptology - CRYPTO '93*, vol. 773. Springer, 1993, pp. 232–249.
- [28] R. Canetti and H. Krawczyk, "Analysis of key-exchange protocols and their use for building secure channels," in *Advances in Cryptology - EUROCRYPT 2001*, vol. 2045. Springer, 2001, pp. 453–474.
- [29] H. Krawczyk, "HMQV: A high-performance secure diffie-hellman protocol," in *Advances in Cryptology - CRYPTO 2005*, vol. 3621. Springer, 2005, pp. 546–566.
- [30] "3GPP TR 33.935 V17.0.0: Study on detailed Long Term Key Update Process (LTKUP)," 2022.
- [31] R. Barbulescu and S. Duquesne, "Updating key size estimations for pairings," *J. Cryptol.*, vol. 32, no. 4, pp. 1298–1336, 2019.

- [32] “3GPP TS 31.102 V17.9.0: Characteristics of the Universal Subscriber Identity Module (USIM) application,” 2023.
- [33] A. Shamir, “How to share a secret,” *Commun. ACM*, vol. 22, no. 11, pp. 612–613, 1979.
- [34] H. Wang, Y. Zhang, and D. Feng, “Short threshold signature schemes without random oracles,” in *Progress in Cryptology - INDOCRYPT 2005*, vol. 3797. Springer, 2005, pp. 297–310.
- [35] J. Groth and A. Sahai, “Efficient non-interactive proof systems for bilinear groups,” in *Advances in Cryptology–EUROCRYPT 2008: 27th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Istanbul, Turkey, April 13–17, 2008. Proceedings 27*. Springer, 2008, pp. 415–432.
- [36] M. Bellare and G. Neven, “Multi-signatures in the plain public-key model and a general forking lemma,” in *CCS 2006*. ACM, 2006, pp. 390–399.

APPENDIX A

THE k -SCDHI ASSUMPTION ON PAIRINGS

Theorem 2. *The k -SCDHI assumption over pairings hold in the generic group model (GGM).*

Proof. We refer to [21] for a detailed description of the generic group model (GGM). We first show a simulator Sim that simulates the k -SCDHI experiment to the adversary $\mathcal{A}$. The simulator Sim randomly samples $x_0, x_1, \dots, x_k \in \mathbb{Z}_q$, and maintains a list $\mathcal{T}$ (initialized to be empty) during the simulation. The element in $\mathcal{T}$ is in the form of $\mathcal{T}(s) = (\xi_s^{(1)}, \xi_s^{(2)}, \xi_s^{(3)})$, where $s \in \mathbb{Z}_q$ denotes the exponent, and $\xi_s^{(1)}, \xi_s^{(2)}, \xi_s^{(3)} \in \{0, 1\}^\lambda$ denote the representations of group elements over $\mathbb{G}_1, \mathbb{G}_2$, and $\mathbb{G}_T$, respectively. Sim randomly samples $\xi_0^{(1)}, \xi_0^{(2)}, \xi_0^{(3)} \in \{0, 1\}^\lambda$ and adds $\mathcal{T}(0) = (\xi_0^{(1)}, \xi_0^{(2)}, \xi_0^{(3)})$ to $\mathcal{T}$. It sends $\mathcal{T}(0)$ to $\mathcal{A}$ and simulates oracles as follows.

- **Exponent operation:** When receiving a query $\text{Exp}(s \in \mathbb{Z}_q, \eta \in \{1, 2, 3\})$, Sim checks whether $\mathcal{T}(s)[\eta]$ (the η -th entry of $\mathcal{T}(s)$) exists. If so, Sim returns $\mathcal{T}(s)[\eta]$. Otherwise, Sim randomly samples $\xi^{(\eta)} \leftarrow \{0, 1\}^\lambda$ which is different from all other $\xi^{(\eta)}$ values, adds $\mathcal{T}(s)[\eta] = \xi^{(\eta)}$, and returns $\mathcal{T}(s)[\eta]$.
- **Multiplication/division operation:** When receiving a query $\text{Mul}(\xi_{s_1}^{(\eta)}, \xi_{s_2}^{(\eta)}, \eta)$, Sim retrieves s_1 and s_2 according to the list $\mathcal{T}$ (Sim returns $\perp$ if such records do not exist), compute $s = s_1 + s_2$, and returns $\mathcal{T}(s)[\eta]$ (If $\mathcal{T}(s)[\eta]$ does not exist, Sim creates one by randomly sampling a different $\xi_s^{(\eta)}$). The division query $\text{Div}(\xi_{s_1}^{(\eta)}, \xi_{s_2}^{(\eta)}, \eta)$ is processed analogously.
- **Pairing operation:** When receiving a query $\text{Pair}(\xi_{s_1}^{(1)}, \xi_{s_2}^{(2)})$, Sim retrieves s_1 and s_2 according to the list $\mathcal{T}$ (Sim returns $\perp$ if such records do not exist), compute $s = s_1 s_2$, and returns $\mathcal{T}(s)[3]$ (If $\mathcal{T}(s)[3]$ does not exist, Sim creates one by randomly sampling a different value).
- **Oracle $\mathcal{O}_{BB}$:** When receiving a query $\text{BB}(m_1, \dots, m_k)$, Sim computes $s = \frac{1}{x_0 + \sum_{i=1}^k x_i m_i}$, and returns $\mathcal{T}(s)[1]$ (If $\mathcal{T}(s)[1]$ does not exist, Sim creates one by randomly sampling a different value).
- **Oracle $\mathcal{O}_{DH_i}$:** When receiving a query $\text{DH}_i(\xi_s^{(2)})$, Sim retrieves s according to the list $\mathcal{T}$ (Sim returns $\perp$ if such a record does not exist), computes $s' = s x_i$, and returns $\mathcal{T}(s')[2]$ (If $\mathcal{T}(s')[2]$ does not exist, Sim creates one by randomly sampling a different value).

Let Q denote the total number of $\mathcal{O}_{BB}(\cdot)$ queries made by $\mathcal{A}$. From the simulation above we know that, all possible representations of group elements in $\mathbb{G}_1$ the adversary can obtain are in the form of

$$s = \alpha + \sum_{j \in [Q]} \frac{\beta_j}{x_0 + \sum_{i=1}^n x_i m_{j,i}}, \quad (4)$$

where the constants α, β_j are chosen by $\mathcal{A}$. Therefore, for a fresh message vector $(m_1^*, \dots, m_k^*)$, $\mathcal{A}$ cannot obtain a representation of group element $\xi_{s^*}^{(1)}$ with $s^* = \frac{1}{x_0 + \sum_{i=1}^k x_i m_i^*}$.

It is still left to argue that the simulation above is correct. There are two cases we have to consider:

Case 1. For a query $\text{BB}(m_1, \dots, m_k)$, we have $\frac{1}{x_0 + \sum_{i=1}^k x_i m_i} = 0$. Since x_i are randomly sampled over $\mathbb{Z}_q$, this happens with probability at most $Q/(q-1)$, where Q is the total query number of $\mathcal{O}_{BB}(\cdot)$ (which is bounded by the running time of $\mathcal{A}$).

Case 2. There are two distinct representations for a single group element. That is, there are s and s' in the form of Eq (4) such that $s - s' = 0$. After reformulating the presentation of $s - s'$, we know $s - s'$ is a polynomial of degree at most $(Q_{BB} + Q_{DH})$ (the total query numbers of $\mathcal{O}_{BB}(\cdot)$ and $\{\mathcal{O}_{DH_i}\}_i$), which again is bounded by the running time of $\mathcal{A}$. Due to the birthday bound, we know Case 2 happens with probability lower than $(Q_{BB} + Q_{DH})^3/(q-1)$. $\square$

APPENDIX B

THE POINTCHEVAL-SANDERS SIGNATURE AND AAKA+PS

Definition 3 (PS-2 Assumption (Assumption 2 in [12])). *The Pointcheval-Sanders-2 (PS-2) assumption holds over a pairing group Φ , if given $(g_2^{x_0}, g_2^{x_1})$ with randomly sampled $x_0, x_1 \in \mathbb{Z}_q$ and an oracle $\mathcal{O}(\cdot)$, it is hard to output a pair $(h, h^{x_0+x_1 m^*})$ for a fresh m^* not being queried to $\mathcal{O}$. Here the oracle $\mathcal{O}(\cdot)$ takes as input m and outputs $(h, h^{x_0+x_1 m})$ with $h \in \mathbb{G}_1$ a randomly sampled element.*

We recall the Pointcheval-Sanders (PS) (multi-message) signature scheme [12], which is EUF-CMA secure under the PS-2 assumption.

- **KeyGen(1^λ):** Let $\Phi = (q, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, g_1, g_2, e)$ be a pairing group. Randomly sample $x_0, x_1, \dots, x_k \in \mathbb{Z}_q$, and compute $X_0 = g_2^{x_0}, X_1 = g_2^{x_1}, \dots, X_k = g_2^{x_k}$. Return $(\text{pk}, \text{sk}) = ((X_0, \dots, X_k), (x_0, \dots, x_k))$.
- **Sign($\text{sk}, (m_1, \dots, m_k)$):** Randomly sample $\sigma_1 \in \mathbb{G}_1^*$, and compute $\sigma_2 = \sigma_1^{x_0 + \sum_{i=1}^k x_i m_i}$. Return (σ_1, σ_2) .
- **Ver($\text{pk}, (m_1, \dots, m_k), (\sigma_1, \sigma_2)$):** Return 1 if $e(\sigma_1, X_0 \prod_{i=1}^k X_i^{m_i}) = e(\sigma_2, g_2)$, and 0 otherwise.

Now we propose AAKA+PS, which enjoys shorter sizes of credentials and (anonymous) proofs. AAKA+PS is the same as AAKA+BB, except that we apply the PS signature scheme [12] as the underlying building block instead of the (variant) BB signature scheme [9], [11]. Due to the similarity we only highlight the different parts.

- **Credential Issue :** $\text{cred} \leftarrow \text{CredIssue}(\text{ipk}, \text{isk}, m, pm)$.

Let $\text{isk} = (x_0, x_1, x_2)$. HN samples $\sigma_1 \in \mathbb{G}_1$, generates a PS signature $(\sigma_1, \sigma_2 = \sigma_1^{x_0+x_1m+x_2pm})$, and a zero-knowledge proof

$$\pi_2 \in \text{ZKPoK}\{(x_0, x_1, x_2) : \sigma_2 = \sigma_1^{x_0+x_1m+x_2pm} \bigwedge_{i=0}^2 X_i = g_2^{x_i}\}.$$

Then HN sends (m, pm) and $\text{cred} = (\sigma_1, \sigma_2, \pi_2)$ to USIM.

• **ProofShow** : $\sigma \leftarrow \text{ProofShow}(\text{ipk}, \text{tpk}, m, pm, \text{cred})$.

let $\text{cred} = (\sigma_1, \sigma_2, \pi_2)$. UE randomly samples $r, t, u \in \mathbb{Z}_q^*$, and computes

- $\hat{\sigma}_1 = \sigma_1^r \in \mathbb{G}_1$, $\hat{\sigma}_2 = \sigma_2^r \cdot \hat{\sigma}_1^t \in \mathbb{G}_1$,
- $C_1 = X_0 X_1^m X_2^{pm} g_2^t \in \mathbb{G}_2$,
- $C_2 = g_2^u \in \mathbb{G}_2$,
- $C_3 = Z^u \cdot g_2^{pm} \in \mathbb{G}_2$,
- $C_4 = h^{pm} \in \mathbb{G}_1$, where $h = H_2(\hat{\sigma}_1, \hat{\sigma}_2, C_1, C_2, C_3, m) \in \mathbb{G}_1$.

$$\pi_3 \in \text{ZKPoK}\{(pm, t, u) : C_1 = X_0 X_1^m X_2^{pm} g_2^t \wedge C_2 = g_2^u \wedge C_3 = Z^u \cdot g_2^{pm} \wedge C_4 = h^{pm}\}.$$

UE sets $\text{pf} = (\hat{\sigma}_1, \hat{\sigma}_2, C_1, C_2, C_3, C_4, \pi_3)$. Finally, it sends (m, pf) to MNO.

• **ProofVer** : $0/1 \leftarrow \text{ProofVer}(\text{ipk}, \text{tpk}, m, \text{pf})$.

Upon receiving $(m, \text{pf} = (\hat{\sigma}_1, \hat{\sigma}_2, C_1, C_2, C_3, C_4, \pi_3))$, SN executes as follows.

1. Check the validity of UE's current subscription based on the information of m . If the subscription has expired, SN rejects the registration request.

2. Identify the HN's public key and retrieve the relevant $\text{ipk} = (X_0, X_1, X_2)$.

3. Check whether $e(\hat{\sigma}_1, C_1) = e(\hat{\sigma}_2, g_2)$.

4. Check the validity of π_3 . The detail of the zero-knowledge scheme is shown in Appendix C.

Theorem 3. *If the SXDH assumption holds in $\mathbb{G}_1$ and $\mathbb{G}_2$, the PS signature scheme is unforgeable, and π_3 is a proof of knowledge scheme, then AAKA+PS is a secure AAKA+ scheme, i.e., it has unforgeability, unlinkability, traceability, and pseudorandomness of session keys.*

Proof. The proof is roughly similar to the proof of AAKA+BB in Section VI, so we only give a sketch and point out the differences here.

1) Unforgeability. Similar as before, when $\mathcal{A}$ finally outputs $(m, \text{pf} = (\hat{\sigma}_1, \hat{\sigma}_2, C_1, C_2, C_3, C_4, \pi_3))$ in the unforgeability experiment, we first extract a witness (pm, t, u) such that

$$C_1 = X_0 X_1^m X_2^{pm} g_2^t \wedge e(\hat{\sigma}_1, C_1) = e(\hat{\sigma}_2, g_2).$$

Therefore, $(\hat{\sigma}_1, \hat{\sigma}_2 / (\hat{\sigma}_1^t))$ is a valid PS signature for message (m, pm) , and we can reduce the unforgeability of AAKA+PS to the unforgeability of the PS signature scheme.

Another thing is to design a simulator to simulate a proof pf' when $\mathcal{A}$ asks $\text{PROOF}(m, pm)$. The simulator first randomly

samples $\alpha, \beta, u \in \mathbb{Z}_q^*$, and computes $\hat{\sigma}_1 = g_1^\alpha$, $\hat{\sigma}_2 = g_1^{\alpha\beta}$, and $C_1 = g_2^\beta$. It is easy to see that $\hat{\sigma}$ and C_1 are uniformly distributed as before, and C_1 is uniquely determined by $\hat{\sigma}_1$ and $\hat{\sigma}_2$ due to the verification equation $e(\hat{\sigma}_1, C_1) = e(\hat{\sigma}_2, g_2)$. The left parts of pf' are generated normally with the help of the zero-knowledge simulator of ZKPoK, which concludes the proof of unforgeability.

2) Unlinkability. The proof of unlinkability is exactly the same as that in AAKA+BB.

3) Traceability. The proof of traceability is the same as that in AAKA+BB, except for the format of the proof. We omit the details here.

4) Pseudorandomness of Session Keys. The proof of pseudorandomness of session keys is the same as that in AAKA+BB. $\square$

APPENDIX C

INSTANTIATIONS OF PROOF OF KNOWLEDGE

In this section, we describe the instantiations of the ZKPoK schemes in Section V.

A. Instantiation of π_0

Recall that in the credential issuance part of AAKA+BB we need a zero-knowledge π_0 :

$$\pi_0 \in \text{ZKPoK}\{(x_0, x_1, x_2) : \bigwedge_{i=0}^2 \sigma_i = \sigma^{x_i} \wedge X_i = g_2^{x_i}\}.$$

Let $H_{zk} : \{0, 1\}^* \rightarrow \mathbb{Z}_q^*$ be a hash function. The prover executes the followings.

- 1) For $i = 0, 1, 2$: randomly sample $\rho_i \in \mathbb{Z}_q$, and compute $\text{cmt}_i = \sigma^{\rho_i}$, $\widetilde{\text{cmt}}_i = g_2^{\rho_i}$.
- 2) Compute $ch = H_{zk}(\{\text{cmt}_i, \widetilde{\text{cmt}}_i\}_{i=0,1,2})$.
- 3) Compute $s_i = \rho_i + x_i \cdot ch$ for $i = 0, 1, 2$.
- 4) Return $\pi_0 = (\{\text{cmt}_i, \widetilde{\text{cmt}}_i\}_{i=0,1,2}, s_0, s_1, s_2)$.

Whenever receiving a proof π_0 , the verifier computes $ch = H_{zk}(\{\text{cmt}_i, \widetilde{\text{cmt}}_i\}_{i=0,1,2})$, and returns 1 if

$$\bigwedge_{i=0}^2 \sigma^{s_i} = \text{cmt}_i \cdot \sigma_i^{ch} \wedge g_2^{s_i} = \widetilde{\text{cmt}}_i \cdot X_i^{ch}.$$

The security proof follows the well-known Schnorr identification protocol and Groth-Sahai proof systems [35], and we omit it here.

Note that we can set $\pi_0 = (ch, s_0, s_1, s_2)$ to reduce the size of proofs.

The completeness is straightforward. Now we prove the above scheme is a proof of knowledge scheme, assuming that H_{zk} is a random oracle.

Witness extractability. According to the forking lemma [36], if an adversary successfully generates a valid zero-knowledge proof π_0 with a non-negligible probability, then we can obtain two groups of valid transcripts $(\{\text{cmt}_i, \widetilde{\text{cmt}}_i\}_{i=0,1,2}, s_0, s_1, s_2)$ and $(\{\text{cmt}_i, \widetilde{\text{cmt}}_i\}_{i=0,1,2}, s'_0, s'_1, s'_2)$ with the same commitments but different challenges ch and ch' . Therefore, for

$i = 0, 1, 2$, $\sigma^{s_i} = cmt_i \cdot \sigma_i^{ch}$ and $\sigma^{s'_i} = cmt_i \cdot \sigma_i^{ch'}$, and hence $\sigma^{(s_i - s'_i)(ch - ch')^{-1}} = \sigma_i$. That is, $x_i = (s_i - s'_i)(ch - ch')^{-1}$.

Zero-knowledge. Now we design a zero-knowledge simulator as follows. The simulator first randomly samples $ch \in \mathbb{Z}_q^*$ and $s_i \in \mathbb{Z}_q$ for $i = 0, 1, 2$. Then, it computes $cmt_i = \sigma^{s_i} / \sigma_i^{ch}$ and $\widetilde{cmt}_i = g_2^{s_i} / X_i^{ch}$. Finally, it reprograms the random oracle such that $H_{zk}(\{cmt_i, \widetilde{cmt}_i\}_{i=0,1,2}) = ch$. It is obvious that the distribution of π_1 is identical as the one honestly generated.

B. Instantiation of π_1

In the credential showing phase of AAKA+BB we need a zero-knowledge proof π_1 :

$$\pi_1 \in \text{ZKPoK}\{(pm, t, r, u) : C_1 = X_0 X_1^m X_2^{pm} g_2^t \wedge C_2 = g_1^r \hat{\sigma}^t \wedge C_3 = g_2^u \wedge C_4 = Z^u \cdot g_2^{pm} \wedge C_5 = h^{pm}\},$$

where the witness is (pm, t, r, u) and the instance is $(X_0, X_1, X_2, Z, \hat{\sigma}, C_1, C_2, C_3, C_4, C_5, m)$.

Let $H_{zk} : \{0, 1\}^* \rightarrow \mathbb{Z}_q^*$ be a hash function (modeled as a random oracle). To generate zero-knowledge proof π_1 , the prover first computes $h = H_2(\hat{\sigma}, C_1, C_2, C_3, C_4, m)$, and executes the following step:

- 1) Randomly sample $\rho_1, \rho_2, \rho_3, \rho_4 \in \mathbb{Z}_q$, and compute $cmt_1 = X_2^{\rho_1} \cdot g_2^{\rho_2}$, $cmt_2 = g_1^{\rho_3} \hat{\sigma}^{\rho_2}$, $cmt_3 = g_2^{\rho_4}$, $cmt_4 = Z^{\rho_4} g_2^{\rho_1}$, $cmt_5 = h^{\rho_1}$.
- 2) Compute $ch = H_{zk}(cmt_1, cmt_2, cmt_3, cmt_4, cmt_5, (A, B, \tau))$ (recall that (A, B, τ) is the transcript of the key exchange sub-protocol).
- 3) Compute $s_1 = \rho_1 + ch \cdot pm$, $s_2 = \rho_2 + ch \cdot t$, $s_3 = \rho_3 + ch \cdot r$, and $s_4 = \rho_4 + ch \cdot u$.
- 4) Return $\pi_1 = (cmt_1, cmt_2, cmt_3, cmt_4, cmt_5, s_1, s_2, s_3, s_4)$.

Upon receiving a proof π_1 , the verifier computes $ch = H_{zk}(cmt_1, \dots, cmt_5, (A, B, \tau))$, and returns 1 if

$$X_2^{s_1} g_2^{s_2} = cmt_1 \cdot (C_1 / (X_0 X_1^m))^{ch} \wedge g_1^{s_3} \hat{\sigma}^{s_2} = cmt_2 \cdot C_2^{ch} \wedge g_2^{s_4} = cmt_3 \cdot C_3^{ch} \wedge Z^{s_4} g_2^{s_1} = cmt_4 \cdot C_4^{ch} \wedge h^{s_1} = cmt_5 \cdot C_5^{ch}.$$

The completeness is straightforward.

Now we prove the above scheme is a proof of knowledge (pok) scheme, assuming that H_{zk} is a random oracle.

Witness extractability. According to the forking lemma [36], if an adversary successfully generates a valid zero-knowledge proof π_1 with a non-negligible probability, then we can obtain two groups of valid transcripts $(\{cmt_i\}_{i=1,\dots,5}, s_1, s_2, s_3, s_4)$ and $(\{cmt_i\}_{i=1,\dots,5}, s'_1, s'_2, s'_3, s'_4)$ with the same commitments but different challenges ch and ch' . Therefore, for we can extract the witness as $pm = (s_1 - s'_1)(ch - ch')^{-1}$, $t = (s_2 - s'_2)(ch - ch')^{-1}$, $r = (s_3 - s'_3)(ch - ch')^{-1}$, and $u = (s_4 - s'_4)(ch - ch')^{-1}$.

Zero-knowledge. Now we design a zero-knowledge simulator as follows. The simulator first randomly samples $ch \in \mathbb{Z}_q^*$ and $s_i \in \mathbb{Z}_q$ for $i = 1, 2, 3, 4$. Then, it computes cmt_i according to the verification equations. Finally, it reprograms the random oracle such that $H_{zk}(cmt_1, cmt_2, cmt_3, cmt_4, cmt_5, (A, B, \tau)) = ch$. The distribution of π_1 is identical to the one honestly generated.

C. Instantiation of π_2

This is similar to the instantiation of π_0 .

$$\pi_2 \in \text{ZKPoK}\{(x_0, x_1, x_2) : \sigma_2 = \sigma_1^{x_0 + x_1 m + x_2 pm} \bigwedge_{i=0}^2 X_i = g_2^{x_i}\}.$$

To generate a zero-knowledge proof π_2 , the prover execute the following steps:

- 1) For $i = 0, 1, 2$: randomly sample $\rho_i \in \mathbb{Z}_q$, and compute $cmt = \sigma_1^{\rho_0 + \rho_1 m + \rho_2 pm}$, $\widetilde{cmt}_i = g_2^{\rho_i}$
- 2) Compute $ch = H_{zk}(cmt, \{\widetilde{cmt}_i\}_{i=0,1,2})$
- 3) Compute $s_i = \rho_i + x_i \cdot ch$ for $i = 0, 1, 2$.
- 4) Return $\pi_2 = (cmt, \{\widetilde{cmt}_i\}_{i=0,1,2}, s_0, s_1, s_2)$.

Whenever receiving a proof π_2 , the verifier computes $ch = H_{zk}(cmt, \{\widetilde{cmt}_i\}_{i=0,1,2})$, and returns 1 if

$$\sigma_1^{s_0 + s_1 m + s_2 pm} = cmt \cdot \sigma_2^{ch} \wedge g_2^{s_i} = \widetilde{cmt}_i \cdot X_i^{ch}.$$

The completeness is straightforward. The proof of extractability and zero-knowledge is similar to the proof of π_0 and we omit it here.

D. Instantiation of π_3

In the credential showing part of AAKA+PS we need a zero-knowledge proof π_3 :

$$\pi_3 \in \text{ZKPoK}\{(pm, t, u) : C_1 = X_0 X_1^m X_2^{pm} g_2^t \wedge C_2 = g_2^u \wedge C_3 = Z^u \cdot g_2^{pm} \wedge C_4 = h^{pm}\},$$

where the witness is (pm, t, u) and the instance is $(X_0, X_1, X_2, Z, \hat{\sigma}_1, \hat{\sigma}_2, C_1, C_2, C_3, C_4, m)$.

Let $H_{zk} : \{0, 1\}^* \rightarrow \mathbb{Z}_q^*$ be a hash function (modeled as a random oracle). To generate a zero-knowledge proof π_3 , the prover first computes $h = H_2(\hat{\sigma}_1, \hat{\sigma}_2, C_1, C_2, C_3, m)$, and executes the following step:

- 1) Randomly sample $\rho_1, \rho_2, \rho_3 \in \mathbb{Z}_q$, and compute $cmt_1 = X_2^{\rho_1} g_2^{\rho_2}$, $cmt_2 = g_2^{\rho_3}$, $cmt_3 = Z^{\rho_3} g_2^{\rho_1}$, $C_4 = h^{\rho_1}$.
- 2) Compute $ch = H_{zk}(cmt_1, cmt_2, cmt_3, cmt_4, (A, B, \tau))$.
- 3) Compute $s_1 = \rho_1 + ch \cdot pm$, $s_2 = \rho_2 + ch \cdot t$, and $s_3 = \rho_3 + ch \cdot u$.
- 4) Return $\pi_3 = (cmt_1, cmt_2, cmt_3, cmt_4, s_1, s_2, s_3)$.

Upon receiving a proof π_3 , the verifier computes $h = H_2(\hat{\sigma}_1, \hat{\sigma}_2, C_1, C_2, C_3, m)$, $ch = H_{zk}(cmt_1, cmt_2, cmt_3, cmt_4)$, and returns 1 if

$$X_2^{s_1} g_2^{s_2} = cmt_1 \cdot (C_1 / (X_0 X_1^m))^{ch} \wedge g_2^{s_3} = cmt_2 \cdot C_2^{ch} \wedge Z^{s_3} g_2^{s_1} = cmt_3 \cdot C_3^{ch} \wedge h^{s_1} = cmt_4 \cdot C_4^{ch}$$

The completeness is straightforward.

Witness extractability. Similar as before, thanks to the forking lemma [36], if an adversary successfully generates a valid zero-knowledge proof π_3 with a non-negligible probability, then we can obtain two group of valid transcripts $(\{cmt_i\}_{i=1,2,3,4}, s_1, s_2, s_3)$ and $(\{cmt_i\}_{i=1,2,3,4}, s'_1, s'_2, s'_3)$ with the same commitments but different challenges ch and ch' . Therefore, for we can extract the witness as $pm =$

$(s_1 - s'_1)(ch - ch')^{-1}$, $t = (s_2 - s'_2)(ch - ch')^{-1}$, and $u = (s_3 - s'_3)(ch - ch')^{-1}$.

Zero-knowledge. Now we design a zero-knowledge simulator as follows. The simulator first randomly samples $ch \in \mathbb{Z}_q^*$ and $s_1, s_2, s_3 \in \mathbb{Z}_q$. Then, it computes cmt_i for $i = 1, 2, 3, 4$ according to the verification equations. Finally, it reprograms the random oracle such that $H_{zk}(cmt_1, cmt_2, cmt_3, cmt_4) = ch$. It is obvious that the distribution of π_3 is identical as the one honestly generated.